



WYDZIAŁ FIZYKI TECHNICZNEJ  
I MATEMATYKI STOSOWANEJ

Imię i nazwisko studenta: Krzysztof Strzelecki

Nr albumu: 185511

Poziom kształcenia: studia drugiego stopnia

Forma studiów: stacjonarne

Kierunek studiów: Fizyka Techniczna

Specjalność: Informatyka stosowana

## **PRACA DYPLOMOWA MAGISTERSKA**

Tytuł pracy w języku polskim: Dekompilator wspomagany sztuczną inteligencją jako minimalistyczny przykład odkrywania modelu fizycznego.

Tytuł pracy w języku angielskim: AI-assisted decompiler as a minimalist example of physical model discovery.

Opiekun pracy: dr inż. Bartosz Reichel

## STRESZCZENIE

Celem pracy było zaprojektowanie i ocena dekompilatora wspomaganego sztuczną inteligencją, który przekształca kod maszynowy (w postaci heksadecymalnej) na kod źródłowy w języku C. W tym celu zastosowano modele językowe o architekturze Transformer – zarówno własną implementację, jak i pretrenowane modele T5-small oraz T5-base. Pracę poprzedziła analiza dostępnych narzędzi do inżynierii odwrotnej oraz opis działania modeli językowych z uwzględnieniem ich analogii do odkrywania modeli fizycznych.

Do trenowania przygotowano syntetyczne zbiory danych zawierające pary kod binarny – kod źródłowy. Modele trenowano na zestawach o wielkości od 1000 do 20000 przykładów. Ich skuteczność oceniano przy użyciu trzech metryk: Similarity, BLEU oraz Semantic Similarity. Najlepsze wyniki osiągnął model T5-small przy 20000 przykładach, uzyskując ponad 86% zgodności semantycznej. Wyniki potwierdziły, że AI może skutecznie modelować proces odwrotny do kompilacji, a dekompilacja może być rozumiana jako forma statystycznego odkrywania reguł rządzących przekształceniem danych — analogicznie do odkrywania praw fizycznych.

### **Słowa kluczowe:**

Uczenie maszynowe, sztuczna inteligencja, modele fizyczne, inżynieria wsteczna, Python, programowanie, LLM, kompilacja, dekompilacja

## **ABSTRACT**

The aim of this thesis was to design and evaluate an AI-assisted decompiler capable of transforming machine code (in hexadecimal form) into source code in the C programming language. To achieve this, Transformer-based language models were employed — including both a custom implementation and pre-trained models T5-small and T5-base. The work was preceded by an analysis of existing reverse engineering tools and an explanation of language model mechanisms, with emphasis on their analogy to the process of discovering physical models.

For training, synthetic datasets containing binary–source code pairs were prepared. The models were trained on datasets ranging in size from 1,000 to 20,000 examples. Their performance was evaluated using three metrics: Similarity, BLEU, and Semantic Similarity. The best results were achieved by the T5-small model trained on 20,000 examples, reaching over 86% semantic accuracy. The results confirmed that AI can effectively approximate the inverse process of compilation, and that decompilation can be interpreted as a form of statistical discovery of transformation rules — analogous to discovering physical laws.

### **Key words:**

Machine Learning, Artificial Intelligence, Physical Models, Reverse Engineering, Python, Programming, LLM, Compilation, Decompilation

# SPIS TREŚCI

|                                                                                       |    |
|---------------------------------------------------------------------------------------|----|
| STRESZCZENIE.....                                                                     | 2  |
| ABSTRACT.....                                                                         | 3  |
| WYKAZ WAŻNIEJSZYCH OZNACZEŃ I SKRÓTÓW.....                                            | 6  |
| 1. Wstęp teoretyczny.....                                                             | 7  |
| 1.1 Wstęp i cel pracy.....                                                            | 7  |
| 1.2 Dekompilacja jako przykład odkrywania modelu fizycznego.....                      | 8  |
| 1.3 Kompilacja a dekompilacja – wprowadzenie.....                                     | 9  |
| 1.4 Trudności związane z dekompilacją.....                                            | 14 |
| 1.5 Przegląd narzędzi do dekompilacji.....                                            | 15 |
| 1.6 Podejścia do dekompilacji z wykorzystaniem sztucznej inteligencji.....            | 16 |
| 1.7 Modele językowe.....                                                              | 17 |
| 1.8 Fizyczne podstawy działania dużych modeli językowych (LLM).....                   | 20 |
| 1.8.1 Tokenizacja – tekst jako liczby.....                                            | 21 |
| 1.8.2 Embedding i un-embedding – odwzorowanie tokenów.....                            | 21 |
| 1.8.3 Dodanie pozycji – pozycjonowanie (Position Encoding).....                       | 23 |
| 1.8.4 Mechanizm Self-Attention (Skalowana uwaga punktowa) i warstwa feed-forward..... | 24 |
| 1.8.5 Enkoder i dekodery w architekturze Transformer.....                             | 26 |
| 1.8.6 Wykonanie fizyczne – jak działa LLM.....                                        | 27 |
| 2. Projekt i implementacja dekompilematoru.....                                       | 29 |
| 2.1 Cel i zakres implementacji.....                                                   | 29 |
| 2.2 Architektura systemu.....                                                         | 29 |
| 2.3 Przygotowanie danych uczących.....                                                | 32 |
| 2.3.1 Generowanie kodu źródłowego.....                                                | 32 |
| 2.3.2 Kompilacja i ekstrakcja sekcji _text.....                                       | 33 |
| 2.3.3 Tworzenie par HEX → C.....                                                      | 35 |
| 2.3.4 Format i struktura danych.....                                                  | 35 |
| 2.4 Wykorzystanie modelu językowego.....                                              | 35 |
| 2.4.1 Klasyczny Transformer.....                                                      | 36 |
| 2.4.2 T5-small (pretrenowany Transformer typu encoder-decoder).....                   | 36 |
| 2.4.3 T5-base (rozszerzona wersja T5).....                                            | 37 |
| 2.4.4 Proces trenowania i ewaluacji.....                                              | 37 |
| 2.5 Analiza zbioru danych uczących.....                                               | 38 |
| 2.5.1 Rozmiary i warianty zbiorów.....                                                | 38 |
| 2.5.2 Charakterystyka danych.....                                                     | 38 |
| 2.5.3 Długość sekwencji i tokenizacja.....                                            | 39 |
| 2.6 Przetwarzanie danych wejściowych.....                                             | 39 |
| 2.6.1 Tokenizacja.....                                                                | 39 |
| 2.6.2 Attention mask i padding.....                                                   | 40 |
| 2.6.3 Format danych przekazywanych do modelu.....                                     | 40 |
| 3. Eksperymenty i wyniki.....                                                         | 42 |
| 3.1 Konfiguracja eksperymentów i środowiska testowego.....                            | 42 |
| 3.1.1 Trenowanie modelu Transformer.....                                              | 43 |
| 3.1.3 Trenowanie modelu T5-base.....                                                  | 49 |
| 3.2 Metody oceny jakości dekompilacji.....                                            | 51 |
| 3.2.1 Similarity score – dopasowanie ciągów znaków.....                               | 51 |

|                                                         |    |
|---------------------------------------------------------|----|
| 3.2.2 BLEU score – zgodność tokenów.....                | 52 |
| 3.2.3 Semantic similarity – zgodność logiczna kodu..... | 52 |
| 3.2.4 Uzasadnienie wyboru metryk.....                   | 53 |
| 3.3 Wyniki dla poszczególnych modeli.....               | 53 |
| 3.3.1 Przebieg treningu modelu Transformer.....         | 54 |
| 3.3.2 Przebieg treningu modelu T5-small.....            | 55 |
| 3.3.3 Przebieg treningu modelu T5-base.....             | 56 |
| 3.3.4 Analiza porównawcza.....                          | 56 |
| 3.4 Wpływ wielkości zbioru danych.....                  | 57 |
| 3.5 Odtwarzanie modelu fizycznego z danych.....         | 60 |
| 3.6 Testowanie na zawężonym zbiorze przypadków.....     | 63 |
| 3.7 Wnioski i obserwacje.....                           | 64 |
| 4. Podsumowanie.....                                    | 67 |
| WYKAZ LITERATURY.....                                   | 69 |
| Wykaz rysunków.....                                     | 71 |
| WYKAZ TABEL.....                                        | 72 |

## WYKAZ WAŻNIEJSZYCH OZNACZEŃ I SKRÓTÓW

$x$  – wejściowy wektor tokenu

$d$  – wymiar embeddingu (rozmiar wektora cech)

$d_{\text{model}}$  – rozmiar modelu (liczba wymiarów embeddingu)

$d_{\text{ff}}$  – rozmiar wewnętrznej warstwy feed-forward

$V$  – rozmiar słownika (liczba tokenów w vocabulary)

$M$  – macierz embeddingu,  $M \in \mathbb{R}^{V \times d}$

$W_1, W_2$  – macierze wag warstwy feed-forward

$b_1, b_2$  – wektory przesunięcia warstwy feed-forward

$W^Q, W^K, W^V$  – macierze wag (projekcji) dla zapytań (query), kluczy (key) i wartości (value)

$Q, K, V$  – macierze zapytań, kluczy i wartości w mechanizmie uwagi

$\sigma$  – funkcja aktywacji nieliniowej

softmax – funkcja normalizująca

FFN( $x$ ) – wynik działania warstwy feed-forward

Attention( $Q, K, V$ ) – wynik mechanizmu uwagi

$e_i$  – wektor one-hot reprezentujący token  $i$

Embed( $i$ ) – embedding tokenu  $i$

UnEmbed( $x$ ) – rozkład prawdopodobieństwa dla tokenów

AI - Artificial Intelligence – sztuczna inteligencja

BPE - Byte Pair Encoding – kodowanie par bajtów (metoda tokenizacji)

CNN - Convolutional Neural Network – konwolucyjna sieć neuronowa

CPU - Central Processing Unit – jednostka centralna (procesor)

ELF - Executable and Linkable Format – format plików binarnych w systemach Linux

FFN - Feed-Forward Network – warstwa w pełni połączona w modelu transformera

FP8 - 8-bitowa precyzja zmiennoprzecinkowa (format stosowany w GPU)

GELU - Gaussian Error Linear Unit – funkcja aktywacji używana w LLM

GPU - Graphics Processing Unit – procesor graficzny

IDA - Interactive DisAssembler – narzędzie do analizy binarnej

LLM - Large Language Model – duży model językowy

NLP - Natural Language Processing – przetwarzanie języka naturalnego

PE - Portable Executable – format plików binarnych w systemie Windows

ReLU - Rectified Linear Unit – funkcja aktywacji w sieciach neuronowych

RNN - Recurrent Neural Network – rekurencyjna sieć neuronowa

T5 - Text-to-Text Transfer Transformer – model językowy Google Research

TPU - Tensor Processing Unit – akcelerator AI opracowany przez Google

UNK - Unknown – specjalny token oznaczający nieznany symbol

WASM - WebAssembly – format binarny do uruchamiania kodu w przeglądarce

# 1. WSTĘP TEORETYCZNY.

## 1.1 Wstęp i cel pracy.

Wraz z dynamicznym rozwojem technologii informatycznych oraz powszechną cyfryzacją wielu aspektów życia, rośnie zapotrzebowanie na narzędzia umożliwiające analizę i przetwarzanie kodu maszynowego. Jednym z kluczowych wyzwań w tym zakresie pozostaje dekompilacja, czyli proces odwrotny do kompilacji, polegający na odtwarzaniu kodu źródłowego na podstawie pliku wykonywalnego. Choć cel ten może wydawać się prosty w założeniach, dekompilacja stanowi złożony problem inżynierski i teoretyczny, wymagający specjalistycznej wiedzy z zakresu architektury komputerów, języków programowania oraz analizy binarnej. Tradycyjne podejścia do dekompilacji opierają się na analizie strukturalnej kodu maszynowego, wykorzystując reguły heurystyczne, disassembler, a także wiedzę ekspercką w celu rekonstrukcji funkcjonalności programu. Jednakże metody te mają ograniczoną skuteczność, szczególnie w przypadku kodu poddanego optymalizacji, obfuskacji lub kompilowanego bez symboli debugowania. W ostatnich latach, wraz z rozwojem dziedziny sztucznej inteligencji oraz dostępnością dużych modeli językowych, pojawiły się nowe możliwości w zakresie automatycznej analizy kodu binarnego. Modele te, pierwotnie projektowane do zadań przetwarzania języka naturalnego [1][2], coraz częściej znajdują zastosowanie również w dziedzinie inżynierii wstecznej [3][4]. Dzięki zdolnościom do reprezentowania i transformowania złożonych sekwencji, umożliwiają one podejście do dekompilacji w sposób statystyczny – traktując ją jako proces tłumaczenia między „językami”: od języka maszynowego do języka programowania wysokiego poziomu. Tego typu podejście zaprezentowano m.in. w badaniach nad LLM4Decompilation [3], DeepRecompiler oraz HexCode [5], które pokazują, że modele językowe mogą efektywnie rekonstruować strukturę kodu źródłowego na podstawie danych binarnych, bez konieczności posiadania wiedzy o strukturze wykonawczej pliku. Celem niniejszej pracy jest zaprojektowanie oraz implementacja dekompileatora wspomagane go sztuczną inteligencją, którego zadaniem jest przekształcenie kodu maszynowego w postaci binarnej na kod źródłowy w języku C. Kluczowym elementem rozwiązania jest wykorzystanie modelu językowego T5 (Text-to-Text Transfer Transformer), który zostaje wytrenowany na odpowiednio przygotowanym zbiorze danych zawierającym pary: kod maszynowy (w formie heksadecymalnej) oraz odpowiadający mu kod źródłowy. Zamiast klasycznego podejścia opartego na regułach, projektowane rozwiązanie traktuje dekompilację jako problem translacji sekwencji wejściowej na sekwencję wyjściową. Dzięki temu możliwe staje się „nauczenie” modelu sztucznej inteligencji odwzorowania zależności pomiędzy binarną reprezentacją programu a jego źródłem logicznym. Zakres pracy obejmuje zarówno część teoretyczną, jak i praktyczną. W części teoretycznej omówione zostaną zagadnienia związane z procesem dekompilacji, trudnościami jakie się z nim wiążą, oraz aktualnym stanem wiedzy w zakresie

wykorzystania metod sztucznej inteligencji w analizie kodu binarnego. Przedstawione zostaną również podstawowe informacje dotyczące architektury modeli językowych typu transformer, ze szczególnym uwzględnieniem modelu T5. Część praktyczna pracy obejmuje przygotowanie zbioru danych treningowych, dostosowanie modelu T5 do zadania dekompilacji, przeprowadzenie procesu uczenia oraz ocenę skuteczności uzyskanego rozwiązania. Na zakończenie pracy zawarte zostaną wnioski oraz propozycje dalszego rozwoju projektu.

## **1.2 Dekompilacja jako przykład odkrywania modelu fizycznego.**

Jednym z najbardziej intrygujących aspektów zastosowania sztucznej inteligencji w dekompilacji jest to, że proces ten można potraktować jako przykład odkrywania modelu fizycznego. W klasycznym ujęciu nauki model fizyczny to uproszczony, matematyczny opis rzeczywistości, który powstaje w wyniku analizy danych eksperymentalnych i obserwacji. Fizyk, analizując trajektorie obiektów, zbiory pomiarów czy wyniki eksperymentów, stara się zrekonstruować reguły rządzące zjawiskami — najczęściej bez bezpośredniego dostępu do "praw natury", a jedynie poprzez indukcyjne wyprowadzanie ogólnych zasad z danych. Analogicznie, w niniejszej pracy dekompilemator wspomagany sztuczną inteligencją został potraktowany jako system uczący się odwrotności działania kompilatora — funkcji, która sama w sobie jest jawna, ale w tej analizie pozostaje ukryta. Model językowy (np. Transformer lub T5) nie jest zasilany regułami gramatyki języka C ani logiką procesów kompilacyjnych. Zamiast tego obserwuje jedynie pary danych wejściowych i wyjściowych: reprezentację binarną programu (HEX) oraz odpowiadający mu kod źródłowy. Na podstawie tych przykładów model „odgaduje”, jaka reguła transformacji leży u podstaw tej relacji. Takie postępowanie jest zgodne z ogólną koncepcją odkrywania funkcji fizycznej — modelu, który tłumaczy obserwowany stan rzeczy. W klasycznym ujęciu kompilator to funkcja:

$$f : \text{Kod źródłowy} \rightarrow \text{Kod maszynowy}$$

Zaś dekompilacja mogłaby być rozumiana jako odwrotność tej funkcji:

$$f^{-1} : \text{Kod maszynowy} \rightarrow \text{Kod źródłowy}$$

Problem polega na tym, że  $f^{-1}$  nie istnieje w sensie matematycznym — transformacja kompilacyjna jest w wielu aspektach nieodwracalna: usuwane są komentarze, zmienne zmieniają nazwę na symbole lub adresy, optymalizacje zmieniają kolejność instrukcji. Mimo to, model językowy trenowany na wystarczająco dużej liczbie przykładów jest w stanie wytworzyć aproksymację  $f^{-1}$ , która nie rekonstruuje dokładnie kodu źródłowego, lecz wytwarza jego semantyczny odpowiednik — coś, co zachowuje logikę i strukturę pierwotnego programu. Z



tego powodu traktujemy nasz model dekompiletora jako statystyczne narzędzie odkrywające funkcję transformacyjną, podobnie jak fizycy odkrywają równania ruchu, modele cząsteczek czy rozkłady prawdopodobieństwa. Nie jest to odtworzenie procesu inżynieryjnego (jak w klasycznej inżynierii odwrotnej), lecz proces przypominający odkrywanie praw rządzących czarną skrzynką — funkcją kompilującą tekst na kod maszynowy, której działanie jest tylko częściowo znane. Praca ta wpisuje się więc w szerszy nurt badań, w których sieci neuronowe i modele językowe są używane nie tylko jako narzędzia klasyfikacyjne, ale również jako systemy eksperymentalnego odkrywania modeli — zarówno w naukach przyrodniczych, jak i technicznych. W tym kontekście dekompiletor AI staje się minimalistycznym przykładem takiego podejścia: złożonym, ale w pełni sterowalnym środowiskiem, w którym można badać zachowania modelu, jego generalizację, ograniczenia i dokładność odkrywania odwrotnej transformacji [6].

*Tabela 1: Analogia między procesem fizycznym a trenowaniem dekompiletora AI.*

| <b>Element procesu naukowego</b>           | <b>Odpowiednik w modelu AI (dekompiletorze)</b>                 |
|--------------------------------------------|-----------------------------------------------------------------|
| Zbieranie danych eksperymentalnych         | Zbiór par: kod źródłowy ↔ kod maszynowy (HEX)                   |
| Obserwacja wyników pomiarów                | Analiza skompilowanych fragmentów programu                      |
| Nieznana funkcja opisana przez przyrodę    | Ukryta transformacja kompilatora $f$                            |
| Odkrywanie modelu na podstawie danych      | Trenowanie modelu językowego do odwzorowania $f^{-1}$           |
| Równanie fizyczne (np. $F=ma$ )            | Aproksymacja odwrotnej funkcji kompilatora $f^{-1}$             |
| Model probabilistyczny z szumem pomiarowym | Model AI z funkcją straty i błędem generacyjnym                 |
| Walidacja eksperymentalna                  | Testowanie jakości dekompilacji na danych testowych             |
| Ograniczenia teorii fizycznej              | Nieodwracalność kodu maszynowego (utrata semantyki, obfuskacja) |

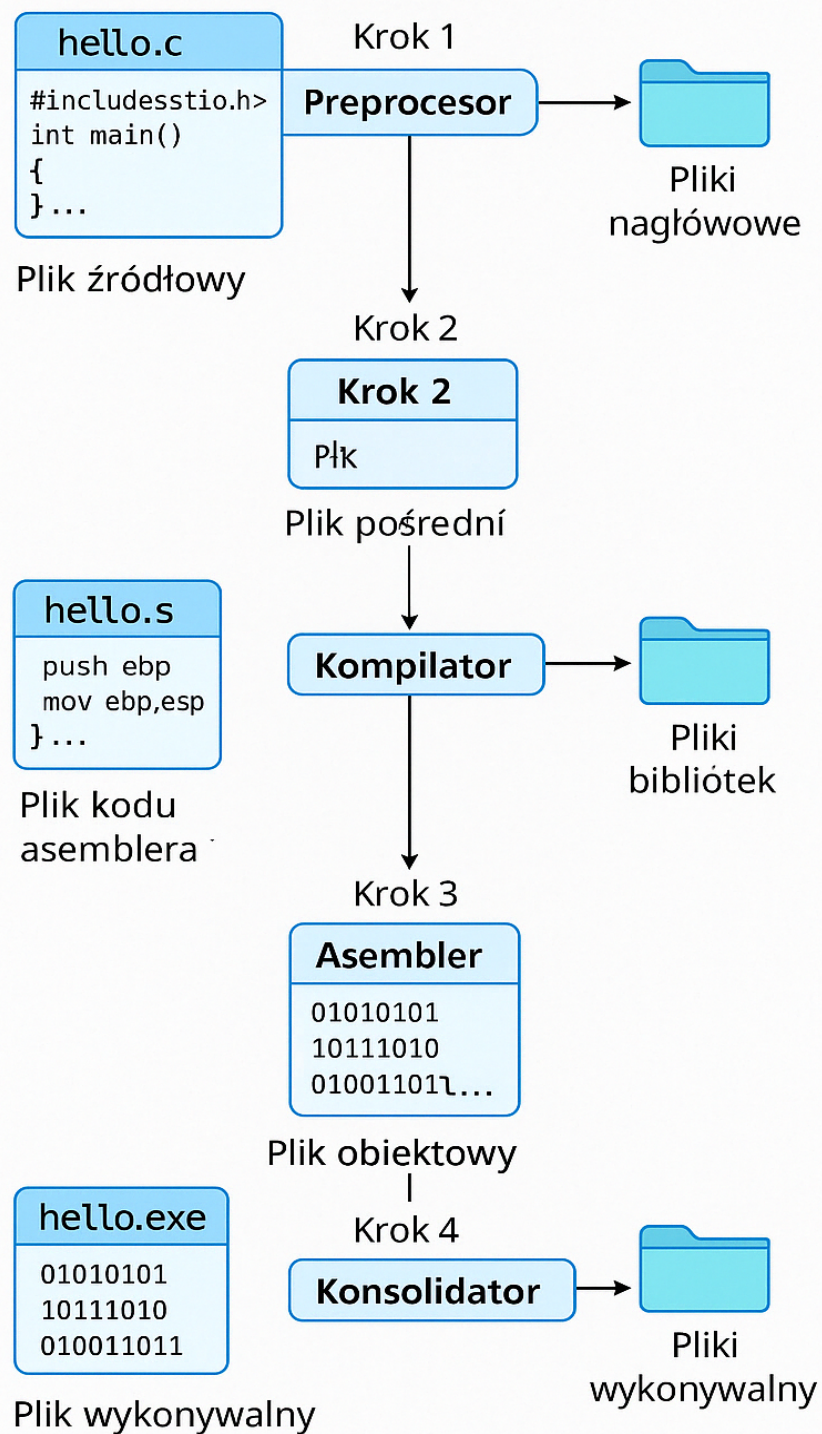
### **1.3 Kompilacja a dekompilacja – wprowadzenie.**

Kompilacja jest procesem tłumaczenia kodu źródłowego napisanego w języku wysokiego poziomu, takim jak C, C++ czy Java, na kod maszynowy zrozumiały dla procesora komputera. Kompilator realizuje tę transformację, generując binarną reprezentację programu (pliki wykonywalne), która może być bezpośrednio interpretowana przez system operacyjny danego środowiska sprzętowego. W trakcie kompilacji stosowane są liczne techniki optymalizacyjne, których celem jest zwiększenie wydajności kodu, zmniejszenie jego rozmiaru, poprawa lokalizacji danych w pamięci, a także eliminacja zbędnych instrukcji [7].

Proces kompilacji w językach takich jak C odbywa się w kilku etapach, które odpowiadają za przekształcanie kodu w sposób strukturalny i semantyczny. Typowy pipeline kompilacji wygląda następująco:

1. Preprocesor – usuwa komentarze, wykonuje dyrektywy preprocesora (np. `#include`, `#define`), rozwija makra.
2. Kompilator – tłumaczy kod C na kod pośredni (np. assembler), optymalizuje wyrażenia, wykrywa błędy składniowe i semantyczne.
3. Asembler – przekształca kod assemblera w kod maszynowy.
4. Linker (konsolidator) – łączy wszystkie moduły (np. biblioteki, pliki obiektowe `.o`) w jeden plik wykonywalny `.exe`, `.out`, `.elf` itd.

Warto zadać pytanie, czy możliwe jest stworzenie dekompiletora, który działałby niemal jak „odwrócony kompilator” – odtwarzając kod źródłowy w sposób jednoznaczny, w pełni czytelny dla człowieka. Choć z technicznego punktu widzenia jest to trudne ze względu na utratę informacji i nieodwracalność wielu transformacji, to rozwój metod opartych na sztucznej inteligencji – w szczególności dużych modeli językowych – otwiera przestrzeń do takich rozważań. Modele te, ucząc się z milionów przykładów, są w stanie rozpoznać wzorce i zależności, które wymykają się klasycznym analizatorom składniowym. W rezultacie, dekompilacja przestaje być wyłącznie procesem inżynieryjnym, a zaczyna przypominać akt rekonstrukcji logicznej i probabilistycznej – zbliżający się coraz bardziej do „odwracalnej kompilacji”.

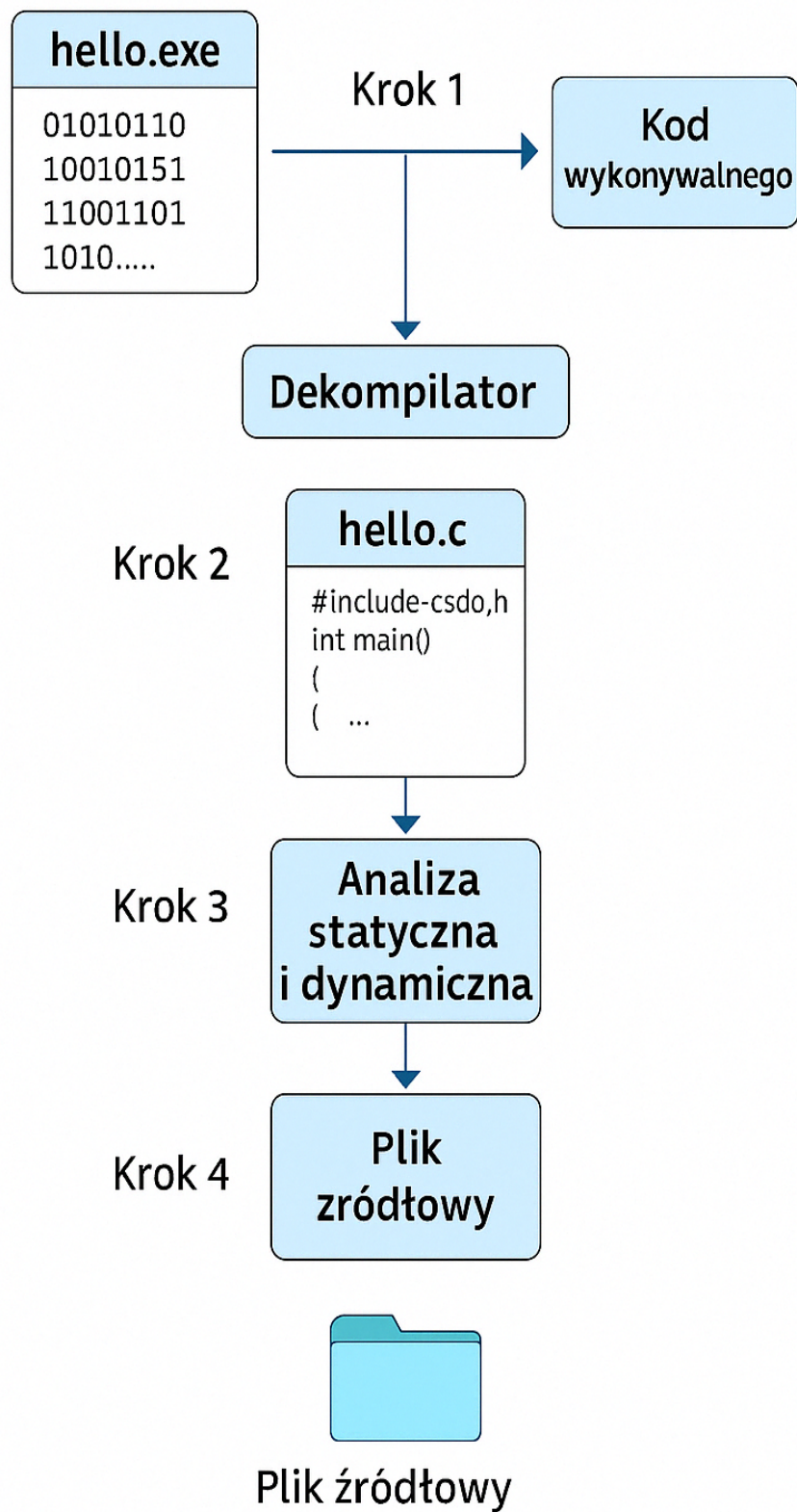


Rys 1: Schemat przedstawia pełen przebieg procesu kompilacji w języku C, włącznie z etapami pośrednimi (.i, .s, .obj) oraz interakcją z plikami nagłówkowymi i bibliotekami.

Dekompilacja natomiast stanowi próbę odtworzenia kodu źródłowego na podstawie jego wersji skompilowanej. W idealnym przypadku proces ten umożliwiłby uzyskanie kodu odpowiadającego pierwotnemu źródłu. W praktyce jednak, z uwagi na nieodwracalność wielu transformacji przeprowadzanych przez kompilator oraz utratę istotnych informacji semantycznych, pełna rekonstrukcja oryginalnego kodu nie jest możliwa. Dekompilacja opiera się więc na rekonstrukcji logicznej struktury programu, często z użyciem heurystyk, analiz statycznych i dynamicznych oraz inżynierii odwrotnej [8]. Proces dekompilacji można podzielić na kilka zasadniczych etapów:

1. Załadowanie pliku binarnego – identyfikacja formatu (np. ELF, PE) oraz podstawowych sekcji (w szczególności `_text`, zawierającej kod wykonywalny).
2. Deasemblacja – przekształcenie kodu maszynowego na instrukcje asemblera, pozwalające na wstępną interpretację działania programu.
3. Analiza przepływu sterowania i danych – rekonstrukcja struktury funkcji, pętli, instrukcji warunkowych oraz zależności między operacjami.
4. Rekonstrukcja struktur wysokiego poziomu – odwzorowanie zmiennych, typów, wywołań funkcji oraz innych konstrukcji językowych.
5. Generowanie pseudokodu – utworzenie przybliżonego kodu źródłowego w języku wysokiego poziomu (np. C), ułatwiającego zrozumienie logiki działania programu.

Pomimo dostępności narzędzi wspomagających ten proces, jakość wygenerowanego kodu zależy od wielu czynników, takich jak poziom optymalizacji, architektura procesora, czy celowe działania obfuskacyjne autora programu.



Rys 2: Schemat procesu dekompilacji: od pliku binarnego, przez deasemblację i analizę kodu, aż do wygenerowania pseudokodu w języku C.



## **1.4 Trudności związane z dekompilacją.**

Proces dekompilacji napotyka liczne przeszkody, które wynikają z natury działania kompilatora oraz z celowego zaciemniania kodu. Jednym z podstawowych problemów jest utrata informacji semantycznej. Podczas kompilacji wiele danych, które są istotne z punktu widzenia czytelności kodu źródłowego, zostaje bezpowrotnie usuniętych. Dotyczy to między innymi nazw zmiennych, funkcji i klas, które w skompilowanej wersji programu zastępowane są adresami pamięci lub symbolami, a często po prostu nie są zapisywane w ogóle. Dodatkowo, komentarze i struktura wizualna kodu – w tym odstępy, podziały na bloki, czy układ logiczny – są całkowicie ignorowane, ponieważ nie wpływają na działanie programu. Tracona jest również jawna informacja o typach danych: podczas kompilacji typy takie jak `int`, `float` czy `struct` przekształcane są w operacje na rejestrach lub pamięci, co czyni ich odtworzenie bardzo trudnym na etapie dekompilacji [9].

Kolejną przeszkodą są optymalizacje wykonywane przez kompilatory. Nowoczesne narzędzia kompilujące stosują szereg technik optymalizacyjnych, które przekształcają kod źródłowy w taki sposób, aby zmaksymalizować jego wydajność. Przykładowo, kompilator może dokonać inliningu, czyli wstawienia treści funkcji bezpośrednio w miejsce jej wywołania, co rozbija modularność kodu. Może również rozwinąć pętle (ang. loop unrolling), aby zminimalizować liczbę iteracji, a także usunąć nieużywany kod (dead code elimination), który nigdy nie zostanie wykonany. Dodatkowo, propagacja stałych może prowadzić do uproszczenia wyrażeń przez zastępowanie zmiennych znanymi wartościami. Wszystkie te działania prowadzą do wygenerowania kodu trudnego do zinterpretowania oraz odległego od oryginalnego źródła pod względem strukturalnym, mimo zachowania tej samej funkcjonalności [10].

Trzecim istotnym wyzwaniem w procesie dekompilacji jest stosowanie technik obfuskacji, czyli zaciemniania kodu. Obfuskacja ma na celu utrudnienie analizy odwrotnej i jest powszechnie wykorzystywana w oprogramowaniu komercyjnym, jak również w złośliwym (malware). Może ona przyjmować różne formy, na przykład zaciemniania przepływu sterowania, w którym wstawiane są dodatkowe, mylące skoki warunkowe zakłócające naturalny przebieg wykonywania programu. Inną metodą jest zmienna alokacja rejestrów, polegająca na losowym przypisywaniu danych do różnych rejestrów, co utrudnia analizę zależności między instrukcjami. Spotykane są również tzw. wstawki kodu nieużywanego (junk code), które zwiększają objętość binarki bez wpływu na jej logikę, jednak znacząco komplikują proces dekompilacji [11].

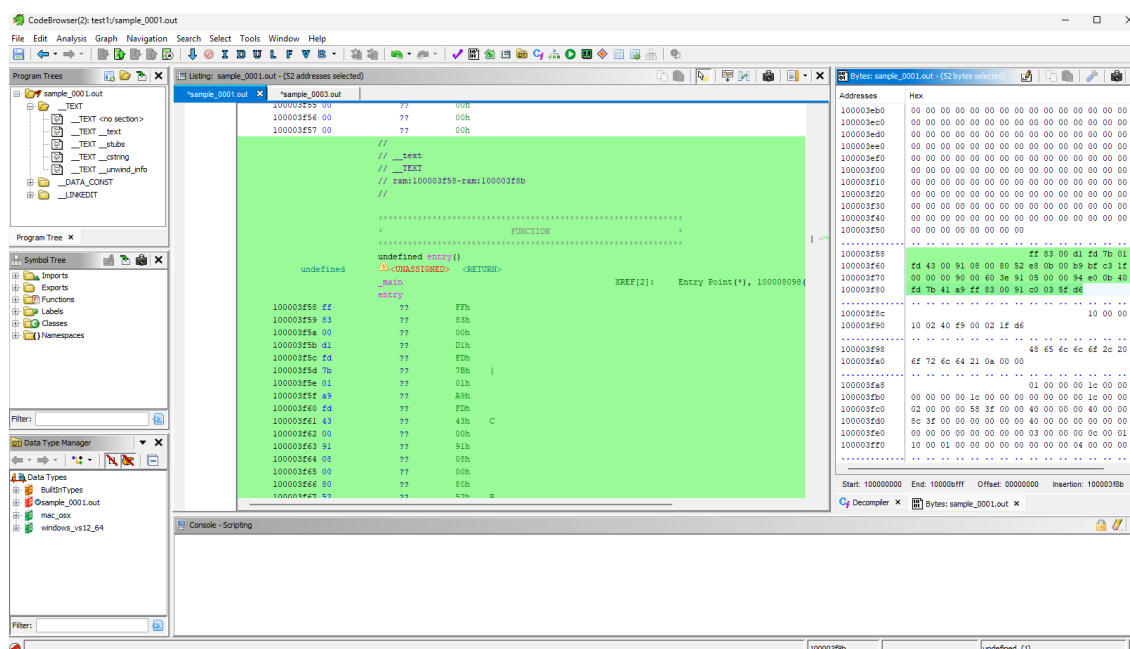
### 1.5 Przegląd narzędzi do dekompilacji.

IDA Pro to jedno z najbardziej znanych narzędzi do analizy binarnej, wyposażone w interaktywny disassembler oraz obsługę wielu architektur procesorów. Wtyczka Hex-Rays umożliwia dekompilację kodu maszynowego do pseudokodu w języku C, znacząco ułatwiając zrozumienie działania programu [12].

Ghidra to zaawansowane narzędzie open-source, opracowane przez amerykańską agencję NSA. Oferuje wsparcie dla licznych architektur, rozbudowany interfejs graficzny, automatyczną dekompilację oraz możliwość integracji z własnymi skryptami analitycznymi [13]. Umożliwia analizę plików wykonywalnych w różnych formatach (m.in. ELF, PE, Mach-O) i identyfikację struktury programu, funkcji, zmiennych oraz przepływu sterowania. Jedną z kluczowych funkcji Ghidry jest jej wbudowany dekompiletor, który przekształca kod maszynowy na pseudokod w stylu języka C. Użytkownik ma również dostęp do widoku asemblera, interaktywnego wykresu przepływu funkcji (*function flow*) oraz narzędzi do śledzenia wywołań i referencji. Wczytując do Ghidry skompilowany plik program.exe, narzędzie automatycznie rozpoznaje format i architekturę binarki, po czym lokalizuje punkt wejścia programu. Po przejściu do głównej funkcji (main) użytkownik ma dostęp do trzech widoków równocześnie:

- kod maszynowy (heksadecymalny),
- instrukcje asemblerowe,
- zrekonstruowany pseudokod w języku C.

Zarówno pseudokod, jak i widok asemblera są zsynchronizowane — kliknięcie w jednej sekcji podświetla odpowiadające fragmenty w pozostałych. Dodatkowo Ghidra umożliwia renaming funkcji i zmiennych, dodawanie komentarzy oraz automatyzację analizy z wykorzystaniem skryptów napisanych w Pythonie lub Javie.



Rys 3: Przykład analizy funkcji main w programie Ghidra.

RetDec to otwarty źródłowy dekompiletor rozwijany przez firmę Avast. Obsługuje wiele formatów plików wykonywalnych oraz architektur, a wyniki dekompilacji prezentowane są w języku C. Umożliwia analizę zarówno lokalną, jak i przez interfejs online [14].

Snowman to szybki i lekki dekompiletor do języka C++, który można wykorzystać samodzielnie lub jako wtyczkę do IDA. Jego główną zaletą jest prostota i szybkość działania, jednak ogranicza się głównie do mniej złożonych przypadków analitycznych [15].

### 1.6 Podejścia do dekompilacji z wykorzystaniem sztucznej inteligencji.

Zastosowanie sztucznej inteligencji, a zwłaszcza dużych modeli językowych, otwiera nowe możliwości w zakresie automatycznej dekompilacji. W tego typu podejściu proces ten traktowany jest jako problem translacji językowej: ciąg bajtów reprezentujących kod maszynowy tłumaczony jest na odpowiadający mu kod źródłowy. Taki sposób rozumowania możliwy jest dzięki rozwojowi tzw. transformerów – modeli sekwencyjnych wykorzystywanych pierwotnie w tłumaczeniu języków naturalnych, ale z powodzeniem adaptowanych do reprezentacji kodu źródłowego [1][16][2]. Dotychczasowe rozwiązania w dziedzinie dekompilacji można podzielić na trzy główne nurty: narzędzia analizy statycznej (np. IDA Pro, Ghidra), metody heurystyczne wykorzystujące reguły wnioskowania, oraz podejścia eksperymentalne z użyciem sztucznej inteligencji. Do tej ostatniej kategorii należą m.in. projekty takie jak DeepRecompiler, SourceTrail [17] czy badania wykorzystujące modele językowe do przewidywania struktury kodu źródłowego na podstawie binariów. Większość tych narzędzi koncentruje się na aspekcie praktycznym: odzyskaniu jak największej ilości czytelnego kodu z jak najmniejszą liczbą błędów.



Zazwyczaj wykorzystują one rozbudowane architektury, duże zbiory danych oraz kompleksowe pre- i post-processory, a ich celem jest budowa pełnoprawnego narzędzia dla inżynierii odwrotnej.

W przeciwieństwie do nich, podejście zaprezentowane w niniejszej pracy ma charakter minimalistyczny i koncepcyjny. Model nie odtwarza dokładnej struktury kodu źródłowego, lecz uczy się odwzorowywać jego funkcjonalny sens. Można go traktować jako eksperymentalny przykład działania modelu językowego, który – podobnie jak badacz w fizyce – próbuje odkryć ukrytą regułę rządzącą transformacją danych. Tym samym dekompilem ten nie tylko rozwiązuje problem inżynierski, ale służy także jako ilustracja procesu odkrywania modelu fizycznego z danych wejściowych.

### **1.7 Modele językowe.**

W ostatnich latach nastąpił gwałtowny rozwój modeli językowych, które pierwotnie stosowane były w zadaniach związanych z przetwarzaniem języka naturalnego (ang. *Natural Language Processing*, NLP). Modele te potrafią analizować, rozumieć i generować tekst, a ich zastosowanie zostało z powodzeniem rozszerzone także na języki programowania. Kod źródłowy – podobnie jak język naturalny – opiera się na regułach gramatycznych i składniowych, dzięki czemu modele językowe mogą być wykorzystywane do jego analizy, refaktoryzacji, tłumaczenia między językami programowania, a także dekompilacji [16]. Podstawą współczesnych modeli językowych jest architektura Transformer, zaprezentowana w pracy *Attention is All You Need* (Vaswani et al., 2017) [2]. Główne cechy tego modelu to:

- możliwość przetwarzania sekwencji równolegle, co znacząco przyspiesza trenowanie,
- mechanizm uwagi (attention), który pozwala modelowi skupić się na najistotniejszych fragmentach danych wejściowych,
- uniwersalność – ten sam mechanizm może być używany do różnych zadań, np. klasyfikacji, tłumaczenia, czy generowania tekstu.

W odróżnieniu od starszych podejść, takich jak rekurencyjne sieci neuronowe (RNN), Transformer operuje na całych sekwencjach naraz i nie wymaga przetwarzania krok po kroku.

Modele językowe dużej skali, znane jako LLM (ang. *Large Language Models*), to współczesne systemy sztucznej inteligencji, które uczą się na ogromnych zbiorach tekstowych w celu modelowania języka, rozumienia semantyki oraz generowania treści. W przeciwieństwie

do wcześniejszych podejść opartych na ręcznie definiowanych regułach lub klasycznych algorytmach statystycznych, LLM są zdolne do uogólniania wiedzy z danych i adaptacji do różnorodnych zadań – często bez konieczności dostrajania. Charakterystyczne cechy LLM to: duża liczba parametrów (od setek milionów do setek miliardów), pretrenowanie na danych ogólnych, często z internetu, repozytoriów kodu, książek i dokumentacji technicznej, transfer learning – możliwość dostosowania do nowych zadań przy minimalnej ilości danych, wszechstronność – mogą być wykorzystywane do klasyfikacji, tłumaczeń, odpowiadania na pytania, streszczania, czy generowania kodu. W ostatnich latach modele takie jak GPT-3, GPT-4, PaLM, LLaMA, BERT, T5, CodeT5, StarCoder czy CodeGen zyskały ogromną popularność, znajdując zastosowanie również poza językiem naturalnym – w programowaniu, analizie kodu, eksploracji danych, a także w cyberbezpieczeństwie.

| Model              | Date    | Provider   | Open-Source | Params | Tunability |
|--------------------|---------|------------|-------------|--------|------------|
| GPT-4 [24]         | 2023.03 | OpenAI     | ✗           | 1.7 T  | ✗          |
| GPT-3.5-turbo      | 2021.09 | OpenAI     | ✗           | 175 B  | ✗          |
| GPT-3 [25]         | 2020.06 | OpenAI     | ✗           | 175 B  | ✗          |
| Cohere-medium [26] | 2022.07 | Cohere     | ✗           | 6 B    | ✓          |
| Cohere-large [26]  | 2022.07 | Cohere     | ✗           | 13 B   | ✓          |
| Cohere-xlarge [26] | 2022.06 | Cohere     | ✗           | 52 B   | ✓          |
| BERT [27]          | 2018.08 | Google     | ✓           | 340 M  | ✓          |
| T5 [28]            | 2019    | Google     | ✓           | 11 B   | ✓          |
| PaLM [29]          | 2022.04 | Google     | ✓           | 540 B  | ✓          |
| LLaMA [3]          | 2023.02 | Meta AI    | ✓           | 65 B   | ✓          |
| CTRL [30]          | 2019    | Salesforce | ✓           | 1.6 B  | ✓          |
| Dolly 2.0 [4]      | 2023.04 | Databricks | ✓           | 12 B   | ✓          |

Rys 4: Porównanie popularnych modeli LLM [18].

Kod źródłowy, choć techniczny, posiada regularną strukturę, gramatykę oraz semantykę, które modele językowe są w stanie uchwycić. Dzięki temu możliwe jest zastosowanie LLM do takich zadań jak:

- generowanie kodu (np. na podstawie opisu funkcji),
- automatyczna dokumentacja,
- wykrywanie błędów,
- tłumaczenie między językami programowania,
- rekonstrukcja kodu na podstawie binariów (czyli dekompilacja).

W przypadku tej pracy wykorzystany został model T5-small, który należy do rodziny LLM, choć jest jej lekką wersją (ok. 60 milionów parametrów). Mimo to, dzięki odpowiedniemu dopasowaniu zadania i danych, nawet mniejsze modele mogą osiągać zadowalające wyniki w

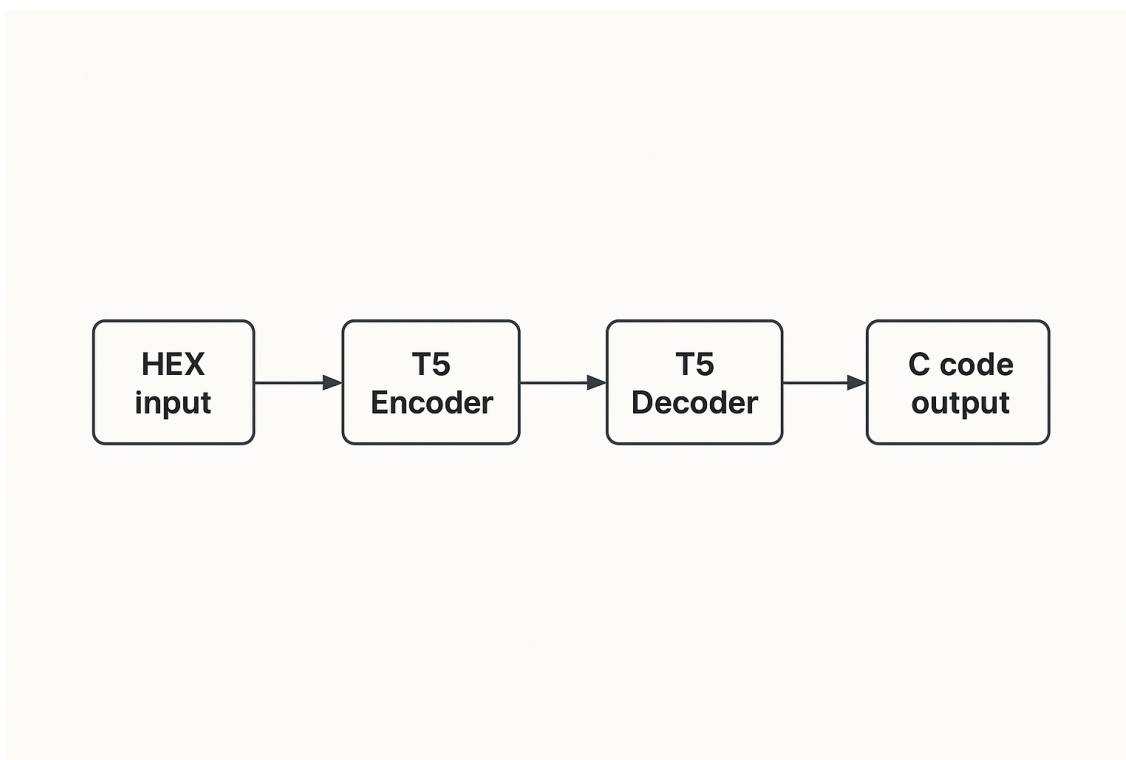
wyspecjalizowanych zastosowaniach. W kontekście dekompilacji, LLM odgrywają szczególnie istotną rolę, ponieważ pozwalają na „rozumienie” i transformację kodu maszynowego w sposób, który byłby trudny do osiągnięcia klasycznymi metodami heurystycznymi. Jednym z reprezentantów modeli językowych dużej skali (LLM) jest T5 – Text-to-Text Transfer Transformer, zaproponowany przez zespół badawczy Google Research. T5 stanowi rozszerzenie architektury Transformer, której cechą wyróżniającą jest ujednolicone podejście do zadań przetwarzania języka: wszystkie problemy – od tłumaczenia i streszczania po odpowiadanie na pytania czy klasyfikację – są traktowane jako translacja tekstu wejściowego na tekst wyjściowy [1]. W przypadku T5 zarówno dane wejściowe, jak i wyjściowe mają postać sekwencji znaków, co czyni ten model niezwykle uniwersalnym. Jego architektura oparta jest na pełnym układzie encoder-decoder, gdzie:

- enkoder koduje sekwencję wejściową do reprezentacji wewnętrznej,
- dekodek generuje sekwencję wyjściową na podstawie tej reprezentacji, token po tokenie.

Rodzina T5 obejmuje wiele wersji różniących się rozmiarem – od t5-small (ok. 60 milionów parametrów), poprzez t5-base, t5-large, aż do t5-11B (ponad 11 miliardów parametrów). W ramach niniejszej pracy wykorzystano model t5-small, ze względu na ograniczenia zasobów obliczeniowych oraz chęć zapewnienia krótkiego czasu treningu. Model T5 można skutecznie wykorzystać do analizy kodu źródłowego, a nawet jego generowania. Dzięki reprezentacji tokenowej i odpowiedniemu trenowaniu, możliwe staje się zastosowanie T5 jako narzędzia do dekompilacji, czyli przekształcania niskopoziomowej reprezentacji binarnej (hex) na kod wysokiego poziomu (C) [18]. W tej konfiguracji T5 działa jako model typu sequence-to-sequence, gdzie:

- wejściem jest sekwencja kodu maszynowego w formacie heksadecymalnym,
- wyjściem – odpowiadający jej kod źródłowy w języku C.

Schemat działania został przedstawiony wcześniej na rysunku:



Rys 5: Schemat działania modelu T5 w konkretnym przypadku.

Tego rodzaju przekształcenie wymaga od modelu nie tylko rozpoznawania wzorców składniowych, ale także rozumienia logiki programistycznej ukrytej w binarnych instrukcjach. Choć `t5-small` nie należy do najbardziej zaawansowanych modeli, jego wyniki w zastosowaniach specjalistycznych są zadowalające, o ile dostarczone dane uczące są dobrze przygotowane.

### **1.8 Fizyczne podstawy działania dużych modeli językowych (LLM).**

Modele językowe dużej skali (LLM) działają w oparciu o architekturę Transformer, której kluczową cechą jest możliwość przetwarzania sekwencji tekstowych jako struktur liczbowych – bez względu na ich długość, strukturę gramatyczną czy język. Cały proces działania modelu można opisać jako przepływ tensora danych przez kolejne warstwy sieci neuronowej, realizowany przez jednostki GPU lub TPU z wykorzystaniem operacji macierzowych.

### 1.8.1 Tokenizacja – tekst jako liczby.

Pierwszym etapem przetwarzania tekstu jest tokenizacja, czyli zamiana tekstu naturalnego na listę indeksów (id) odpowiadających tokenom słownika. Architektura Transformer oraz modele językowe dużej skali (LLM) nie operują bezpośrednio na tekście naturalnym, lecz na danych numerycznych. Dlatego pierwszym krokiem w przetwarzaniu danych wejściowych jest proces tokenizacji, czyli zamiany tekstu na sekwencję liczb reprezentujących podstawowe jednostki językowe. Przykład:

Tekst wejściowy: „return 0;”  
Tokeny: [1532, 11, 32]

Proces ten wykonuje specjalny moduł zwany tokenizerem – odpowiada on za konwersję tekstu na sekwencję liczb na wejściu oraz odwrotną konwersję z liczb na tekst na wyjściu modelu. Tokenizer operuje na zbiorze znanym jako słownik (vocabulary), który zawiera wszystkie możliwe tokeny rozpoznawane przez model. Rozmiar tego zbioru oznaczamy jako:

$$n_{\text{vocabulary}}$$

Wszystkie tokeny wejściowe muszą znajdować się w tym słowniku. Jeśli model napotka fragment tekstu, którego nie zna (czyli token spoza słownika), używa specjalnego tokenu zastępczego – zazwyczaj "[UNK]" (ang. *unknown*) [19]. W praktyce stosuje się różne metody dzielenia tekstu na tokeny, z których najpopularniejsze to:

- Byte Pair Encoding (BPE) – algorytm, który dzieli słowa na najczęściej występujące pary znaków i buduje tokeny na tej podstawie.
- WordPiece – rozwinięcie BPE stosowane m.in. w BERT, tokeny bazują na segmentach słów najczęściej spotykanych w danych treningowych.
- SentencePiece – metoda niezależna od spacji i struktury języka, operująca na poziomie bajtów lub znaków Unicode. Jest wykorzystywana np. w modelach T5 i mT5.

### 1.8.2 Embedding i un-embedding – odwzorowanie tokenów.

W architekturze Transformer tekst nie jest przetwarzany bezpośrednio, lecz zamieniany na liczby. Dlatego kluczową rolę odgrywa warstwa embedding, która umożliwia konwersję każdego tokenu (np. słowa, znaku, fragmentu wyrazu) na reprezentację wektorową w przestrzeni cech.

## Embedding

Każdy token wejściowy jest przekształcany w wektor embeddingowy przy użyciu tablicy odwzorowań – tzw. macierzy embeddingu  $M \in \mathbb{R}^{V \times d}$ , gdzie:  $V$  to rozmiar słownika (liczba możliwych tokenów),  $d$  to długość wektora embeddingowego, często nazywana rozmiarem modelu  $d_{\text{model}}$ . Każdy token reprezentowany jest początkowo jako wektor one-hot  $e_i \in \mathbb{R}^V$ , czyli wektor, który zawiera 1 tylko na pozycji odpowiadającej danemu tokenowi  $i$ , a zera w pozostałych miejscach:

$$e_3 = [0, 0, 0, 1, 0, 0, \dots] \quad (1)$$

Wektor embeddingowy danego tokenu uzyskujemy przez mnożenie:

$$\text{Embed}(i) = e_i \cdot M \quad (2)$$

czyli wybieramy  $i$ -ty wiersz macierzy  $M$ . Wynikiem jest wektor  $x_i \in \mathbb{R}^d$ , który reprezentuje semantyczną przestrzeń tokenu [2].

Dla każdego tokenu  $i$  embedding zostaje wzbogacony o informację o pozycji w sekwencji, przez dodanie pozycyjnego wektora  $p_i$ :

$$z_i = x_i + p_i \quad (3)$$

To połączenie reprezentuje wektory wejściowy do pierwszej warstwy transformera – zawiera zarówno znaczenie tokenu, jak i jego pozycję w zdaniu.

## Un-embedding

Na wyjściu modelu następuje operacja odwrotna – zamiana wektora  $h \in \mathbb{R}^d$  (przewidywania modelu) na prawdopodobieństwo dla każdego tokenu w słowniku. Ten krok realizuje warstwa un-embedding, będąca liniową transformacją z funkcją softmax:

$$\text{UnEmbed}(h) = \text{softmax}(hW + b) \quad (4)$$

gdzie:

$W \in \mathbb{R}^{d \times V}$  - macierz wag (często będąca transpozycją macierzy M),

$b \in \mathbb{R}^V$  - wektor przesunięcia (bias),

wynik to wektor długości V zawierający rozkład prawdopodobieństwa dla każdego tokenu.

Model wybiera jako wynikowy token ten, który ma największe prawdopodobieństwo (top-1) lub losuje go zgodnie z rozkładem (sampling/top-k/top-p). W wielu implementacjach stosuje się technikę tzw. weight tying, polegającą na tym, że macierz un-embeddingu W jest po prostu transpozycją macierzy embeddingu M:

$$W = M^T \quad (5)$$

Takie rozwiązanie pozwala zredukować liczbę parametrów modelu, poprawić spójność między warstwami wejściowymi i wyjściowymi oraz zmniejszyć ryzyko nadmiernego dopasowania (overfitting) [2].

### 1.8.3 Dodanie pozycji – pozycjonowanie (Position Encoding)

Architektura Transformer, w odróżnieniu od rekurencyjnych sieci neuronowych (RNN), nie posiada wbudowanego mechanizmu uwzględniającego kolejność elementów sekwencji. Aby umożliwić modelowi rozróżnienie pozycji tokenów w ciągu wejściowym, do wektorów embeddingowych dodawane są wektory pozycyjne (positional encodings). Są one deterministycznie definiowane jako funkcje sinusoidalno-kosinusowe zależne od pozycji tokenu w sekwencji [18]. Pełna funkcja osadzania pozycji została zaproponowana w pracy Attention is All You Need [2] i zdefiniowana następująco:

$$f(t)_{2k} = \sin\left(\frac{t}{r^k}\right), \quad f(t)_{2k+1} = \cos\left(\frac{t}{r^k}\right), \quad (6),(7)$$

dla  $k = 0, 1, \dots, \frac{d}{2} - 1$

gdzie:

$t \in \mathbb{N}$  oznacza indeks pozycji tokenu w sekwencji,

$d$  to wymiar embeddingu ( liczba cech wektora),

$r = N^{2/d}$ , a  $N$  to hiperparametr (np. 10000 w oryginalnym modelu)

Funkcja pozycyjna  $f(t)$  mapuje każdą pozycję  $t$  na wektor długości  $d$  poprzez naprzemienne użycie funkcji trygonometrycznych:

$$f: \mathbb{N} \rightarrow \mathbb{R}^d$$

Taka konstrukcja pozwala modelowi rozpoznawać relację między pozycjami. Dla przykładu, różnica między wektorami pozycji  $t$  i  $t+\Delta t$  można wyrazić jako liniową transformację:

$$f(t+\Delta t) = \text{diag}(f(\Delta t)) \cdot f(t) \quad (8)$$

Oznacza to, że przesunięcia w sekwencji (np. „jeden token dalej”) można wyrażać jako operacje macierzowe – co ułatwia modelowi uczenie się uwagi zależnej od względnych odległości tokenów (tzw. relative attention). Ponadto, konwolucje, które są popularne w klasycznych sieciach neuronowych (np. CNN), można w tym kontekście zrealizować jako:

$$\sum_j c_j f(t+\Delta t_j) = \left( \sum_j c_j \text{diag}(f(\Delta t_j)) \right) f(t) \quad (9)$$

co stanowi liniową kombinację przesunięć pozycji i pozwala modelowi uczyć się rozkładu uwagi nad sąsiednimi tokenami [2].

#### 1.8.4 Mechanizm Self-Attention (Skalowana uwaga punktowa) i warstwa feed-forward.

Mechanizm self-attention umożliwia każdemu tokenowi w sekwencji ocenę relacji z innymi tokenami. Dla każdego tokenu wejściowego  $x \in \mathbb{R}^d$ , generowane są trzy wektory: zapytania (query), klucze (key) i wartości (value):

$$Q = XW^Q, K = XW^K, V = XW^V \quad (10)$$



gdzie:

$X \in \mathbb{R}^{n \times d}$  to macierz tokenów wejściowych,

$W^Q, W^K, W^V \in \mathbb{R}^{d \times d_k}$  - parametry trenowalne.

Następnie, uwaga(attention) wyliczana jest jako:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (11)$$

Skalowanie przez  $\sqrt{d_k}$  zapobiega nadmiernym wartościom wewnątrz softmaxa, które mogłyby skutkować zanikającymi gradientami. Operacja softmax nadaje większe znaczenie tokenom bardziej istotnym kontekstowo. W praktyce używa się multi-head attention, w którym kilka takich bloków działa równolegle i ich wyniki są łączone:

$$MultiHead(X) = Concat(h_1, \dots, h_h) W^O \quad (12)$$

Każda głowa  $h_i$  to osobny blok attention z własnymi wagami, a  $W^O$  skala ich wyjścia [18].

### Warstwa Feed – Forward

Po każdej warstwie uwagi następuje niezależna dla każdego tokenu transformacja nieliniowa, tzw. feed-forward layer (FFN) [20]. Składa się ona z dwóch warstw liniowych rozdzielonych  $\sigma$  czyli funkcją aktywacji nieliniowej (zazwyczaj ReLU lub GELU):

$$FFN(x) = \sigma(x, XW_1 + b_1) W_2 + b_2 \quad (13)$$

gdzie:

$$W_1 \in \mathbb{R}^{d \times d_{ff}},$$

$$W_2 \in \mathbb{R}^{d_{ff} \times d},$$

$d_{ff}$  to zazwyczaj 4x większy rozmiar niż  $d$  (dla większej pojemności).

### 1.8.5 Enkoder i dekodek w architekturze Transformer

Model Transformer składa się z dwóch głównych komponentów: enkodera oraz dekodera. Oba zbudowane są z powtarzalnych bloków, z których każdy zawiera zestaw precyzyjnie zaprojektowanych warstw odpowiedzialnych za przetwarzanie sekwencji danych. Enkoder odpowiada za analizę wejściowej sekwencji tokenów, natomiast dekodek – za generowanie sekwencji wyjściowej (np. tłumaczenia, wygenerowanego kodu lub zdania).

#### Enkoder

Enkoder w architekturze Transformer przyjmuje jako wejście sekwencję osadzonych tokenów (embeddingów) wzbogaconych o kodowanie pozycyjne. Działa on w pełni równolegle – nie analizuje tokenów sekwencyjnie jak tradycyjne RNN, lecz przetwarza całą sekwencję naraz. Każdy blok enkodera składa się z dwóch głównych warstw:

1. Warstwa self-attention (samozwrotna uwaga) – umożliwia każdemu tokenowi „zwrócenie uwagi” na inne tokeny w sekwencji. Mechanizm ten analizuje związki pomiędzy wszystkimi tokenami równocześnie i nadaje im odpowiednie wagi.
2. Warstwa feed-forward (FFN) – przekształca każdy token niezależnie przy użyciu nieliniowej sieci neuronowej.

Obie warstwy są otoczone przez: mechanizm normalizacji warstwowej (LayerNorm), połączenia rezydualne (residual connections), które ułatwiają przepływ gradientów i stabilizują uczenie. Cały enkoder składa się zwykle z wielu takich identycznych bloków (np. 6, 12 lub 24 w zależności od rozmiaru modelu) [18].

#### Dekoder

Dekoder generuje sekwencję wyjściową (np. tłumaczenie lub kod źródłowy), przetwarzając tokeny „od lewej do prawej” – token po tokenie, z dostępem do wcześniejszych wygenerowanych tokenów oraz informacji z enkodera. Każdy blok dekodera zawiera trzy główne warstwy:

1. Maskowana warstwa self-attention – pozwala tokenowi analizować wyłącznie poprzednie tokeny (nie przyszłe), co zapewnia właściwą kolejność generowania (autoregresyjność). Maskowanie polega na ukrywaniu przyszłych pozycji poprzez nadanie im bardzo niskich wag ( $-\infty$ ) przed softmaxem.

2. Warstwa cross-attention – pozwala dekoderoowi uwzględniać informacje z enkodera. Tokeny generowane przez dekodera mogą dzięki temu „skupić uwagę” na odpowiednich miejscach wejściowej sekwencji.
3. Warstwa feed-forward – taka sama jak w enkoderze, przetwarzająca każdy token niezależnie.

Tak jak w enkoderze, również tutaj każda warstwa jest uzupełniona o połączenie rezydualne i normalizację warstwową. Dekoder produkuje na końcu wektor reprezentujący każdy token wyjściowy, który trafia do warstwy un-embedding w celu wygenerowania rzeczywistego tokenu (np. słowa, znaku, operatora C) [2].

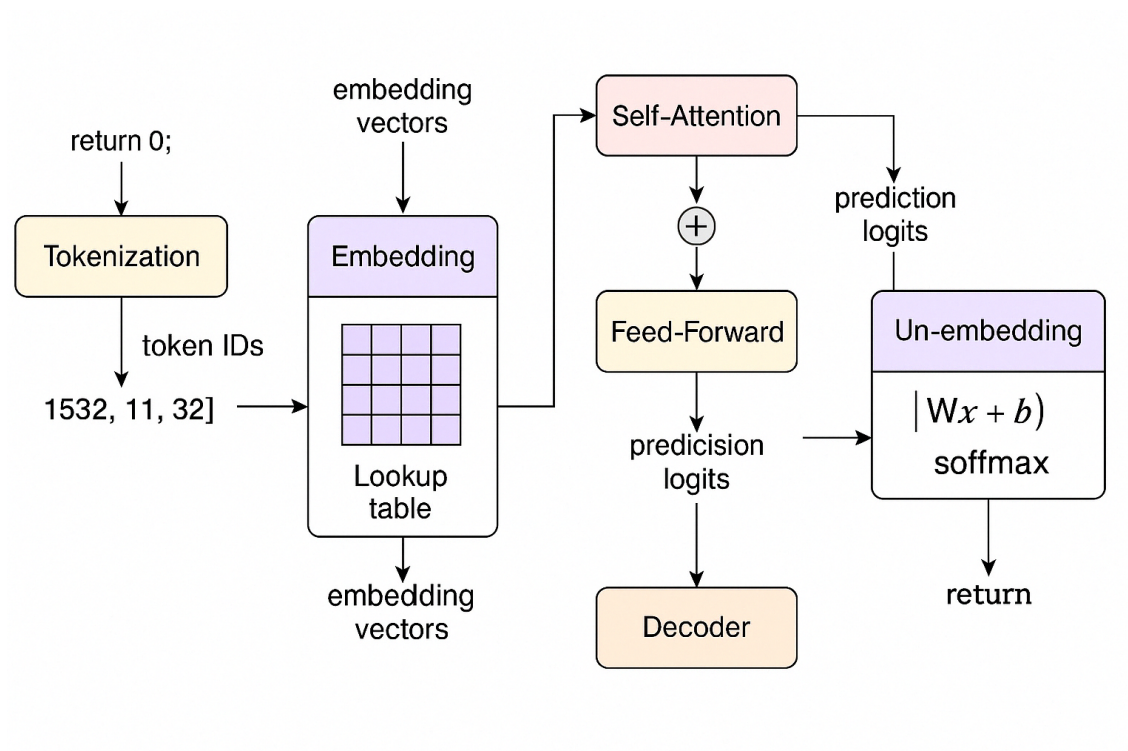
#### *1.8.6 Wykonanie fizyczne – jak działa LLM*

Całość przetwarzania – od tokenizacji, przez embeddingi, po self-attention i generowanie wyjścia – realizowana jest przy użyciu operacji tensorowych i macierzowych, wymagających znacznej mocy obliczeniowej. Z tego względu do działania modeli LLM wykorzystuje się specjalizowany sprzęt:

- GPU (Graphics Processing Units) – szczególnie jednostki NVIDIA z architekturą CUDA, które umożliwiają szybkie mnożenie dużych macierzy i równoległe przetwarzanie dziesiątek tysięcy tokenów jednocześnie.
- TPU (Tensor Processing Units) – akceleratory stworzone przez Google, zoptymalizowane specjalnie pod modele głębokiego uczenia.
- FPGA / ASIC – w zastosowaniach przemysłowych i wbudowanych stosuje się również układy programowalne (FPGA) lub dedykowane (ASIC), zoptymalizowane pod Transformery [21].

Na poziomie technicznym, wszystkie operacje – takie jak mnożenie macierzy, aktywacje, normalizacje, softmax czy propagacja wsteczna (backpropagation) podczas treningu – realizowane są jako równoległe operacje matematyczne nad wielowymiarowymi tensorami. Przykładowo, obliczenie jednej warstwy attention w modelu LLM może wymagać: tysięcy jednoczesnych operacji mnożenia i sumowania, dziesiątek gigabajtów VRAM do przechowywania wag i aktywacji, miliardów parametrów (np. w modelu T5-3B) i setek miliardów operacji dla jednej sekwencji wejściowej [18].

Wszystko to powoduje, że modele LLM są nie tylko zaawansowane koncepcyjnie, ale również niezwykle wymagające obliczeniowo – co stanowi jeden z kluczowych czynników ograniczających ich powszechne zastosowanie.



Rys 6: Schemat przepływu danych w architekturze dużego modelu językowego (LLM) opartego na Transformerze.

## 2. PROJEKT I IMPLEMENTACJA DEKOMPILATORA.

### 2.1 Cel i zakres implementacji.

Celem części praktycznej niniejszej pracy było opracowanie, zaimplementowanie oraz przetestowanie prototypowego dekompilatora wspomagane go sztuczną inteligencją, którego zadaniem jest automatyczne przekształcanie kodu maszynowego – w szczególności w postaci reprezentacji heksadecymalnej – na czytelny dla człowieka kod źródłowy w języku C. Rozwiązanie zostało zaprojektowane jako system sekwencyjny typu sequence-to-sequence, analogiczny do systemów tłumaczenia maszynowego. Podstawową ideą było potraktowanie kodu binarnego (HEX) jako „języka wejściowego”, a odpowiadającego mu kodu źródłowego jako „języka wyjściowego”. Aby przetestować skuteczność podejścia opartego na głębokim uczeniu, zaimplementowano trzy różne modele: klasyczny Transformer oraz dwa warianty modelu T5 – T5-small i T5-base. Modele te zostały wytrenowane na specjalnie przygotowanych parach danych: (wejście: HEX, wyjście: C). Wszystkie modele zaimplementowano w języku Python, przy użyciu bibliotek takich jak PyTorch oraz Hugging Face Transformers. Dodatkowym celem było przeanalizowanie i porównanie efektywności działania poszczególnych modeli oraz przygotowanie uniwersalnego pipeline'u do przetwarzania plików binarnych, ekstrakcji danych, konwersji do formatu wejściowego i automatycznej dekompilacji z wykorzystaniem modelu językowego.

### 2.2 Architektura systemu.

Zaprojektowany system dekompilacji składa się z kilku współpracujących ze sobą komponentów, które razem tworzą kompletny pipeline przetwarzania danych binarnych i generowania kodu źródłowego. Poszczególne elementy systemu można opisać w następujący sposób:

1. **Moduł ekstrakcji danych binarnych:** Pliki wykonywalne w formacie ELF (Linux) lub PE (Windows) są analizowane w celu wyodrębnienia sekcji `_text`, zawierającej instrukcje maszynowe. Sekcja ta zostaje odczytana bajt po bajcie i zapisana w postaci łańcucha znaków HEX (np. "48 83 EC 28"). Na tym etapie wykonywana jest wstępna pre-processingowa filtracja nieistotnych bajtów (np. padding, instrukcje niebędące częścią funkcji).

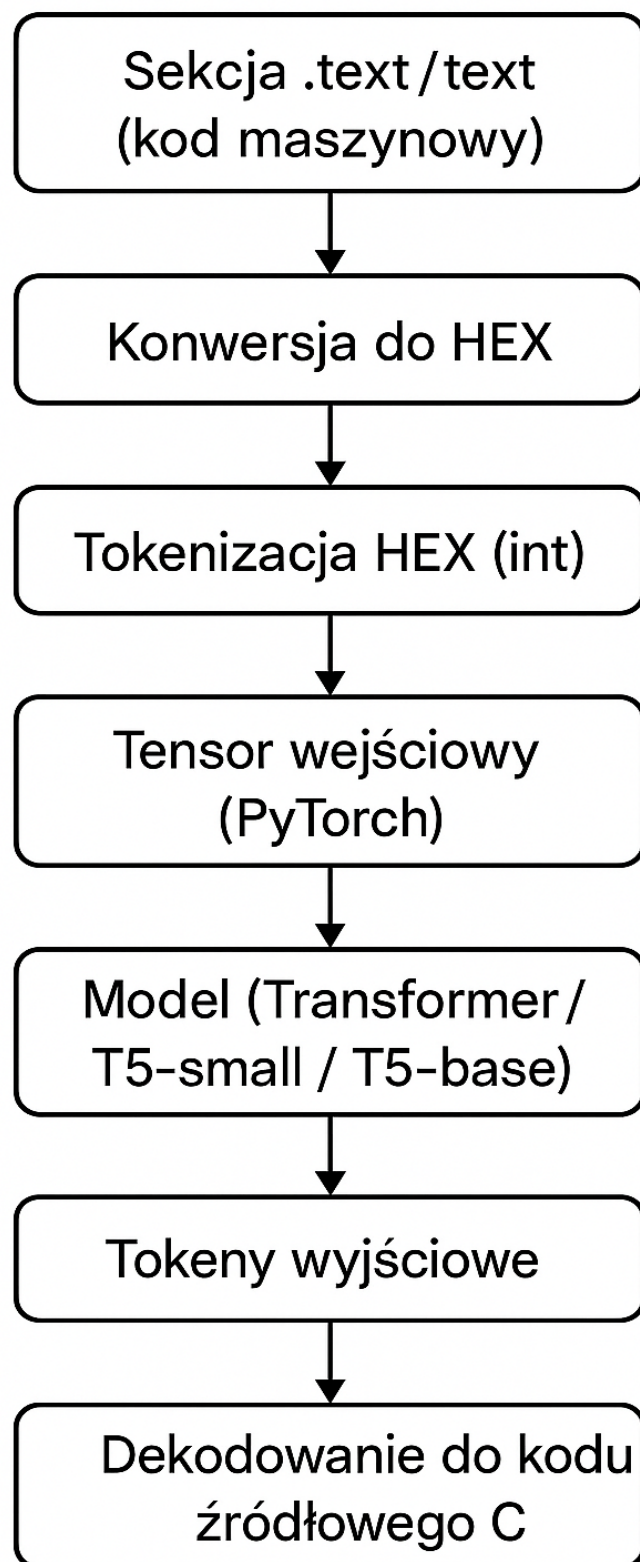
2. **Przygotowanie sekwencji wejściowej:** Dane HEX są konwertowane na sekwencje tokenów liczbowych (np. bajt 48 → int 72) i wczytywane jako tensory `torch.Tensor` o stałej długości (np. 1024 tokeny, z paddingiem). Sekwencje są następnie przekazywane do modelu jako `input_ids`.

3. **Model językowy:** Dane wejściowe są przetwarzane przez jeden z trzech modeli:

1. klasyczny Transformer, zaimplementowany od podstaw w PyTorch z użyciem `torch.nn.Transformer`,
2. model T5-small, załadowany z biblioteki Hugging Face,
3. model T5-base, również załadowany z biblioteki Hugging Face.

Model działa w trybie autoregresyjnym – sekwencja wyjściowa (kod C) generowana jest token po tokenie, a każdy kolejny token zależy od wcześniej przewidzianych [17].

4. **Dekodowanie i postprocessing:** Wygenerowane tokeny są zamieniane na znaki ASCII i łączone w końcowy tekst. Jeśli model zwróci specjalny token końca (`[EOS]` lub `0`), proces generowania zostaje przerwany. Dodatkowo stosowane są reguły wstępnej walidacji kodu (np. zamykanie nawiasów, kończenie średnikami).



Rys 7: Schemat działania dekompileatora wspomagane go sztuczną inteligencją. Diagram przedstawia kolejne etapy procesu: od przetwarzania pliku binarnego i ekstrakcji sekcji `_text`, przez konwersję do formatu HEX i tokenizację, aż po transformację wejściowych danych do postaci kodu źródłowego w języku C za pomocą modelu językowego.

## **2.3 Przygotowanie danych uczących**

Aby wytrenować model językowy zdolny do wykonywania dekompilacji, konieczne było przygotowanie odpowiedniego zbioru danych treningowych, zawierającego pary (HEX → C). Zbiór ten został wygenerowany automatycznie w pełni kontrolowanym środowisku, w celu zapewnienia poprawnych, syntaktycznie jednoznacznych i możliwych do odwzorowania przykładów.

### **2.3.1 Generowanie kodu źródłowego**

Wygenerowano dziesiątki tysięcy losowych, syntetycznych programów w języku C o zróżnicowanej strukturze i złożoności. Do tego celu wykorzystano specjalny skrypt Pythona, który dynamicznie tworzył fragmenty kodu reprezentujące:

- działania arytmetyczne i logiczne,
- instrukcje warunkowe (if, else),
- pętle (for, while),
- funkcje użytkownika i rekurencyjne (factorial, fib, power),
- instrukcje switch-case,
- przykłady z użyciem funkcji printf() z różnymi formatowaniami („%d”, „%s”, „%c”, „%x”).
- proste implementacje matematyczno-fizyczne

Każdy program zapisywany był jako osobno plik .c, nazwany zgodnie z wzorcem „sample\_XXXX.c”



```

38     # 4. Pętla for
39     var = random.choice(var_names)
40     n = random.randint(3, 15)
41     templates.append(f"int main() {{ int sum = 0; for(int {var}=0; {var}<{n}; {var}++; sum += {var}; return sum; }}")
42
43     # 5. Pętla while
44     n = random.randint(2, 10)
45     templates.append(f"int main() {{ int i = 0; while(i < {n}) i++; return i; }}")
46
47     # 6. Switch-case
48     val = random.randint(1, 3)
49     templates.append(f"int main() {{ int x = {val}; switch(x) {{ case 1: return 10; case 2: return 20; default: return 0; }} }}")
50
51     # 7. Operacje bitowe
52     a, b = random.sample(values, 2)
53     templates.append(f"int main() {{ int a = {a}, b = {b}; return a & b; }}")
54
55     # 8. Rekurencyjne: factorial
56     n = random.randint(3, 6)
57     templates.append(f"""
58 int factorial(int n) {{
59     if (n <= 1) return 1;
60     return n * factorial(n - 1);
61 }}
62 int main() {{
63     return factorial({n});
64 }}
65 """)
66
67     # 9. Rekurencyjne: fibonacci
68     n = random.randint(3, 10)
69     templates.append(f"""
70 int fib(int n) {{
71     if (n <= 1) return n;
72     return fib(n - 1) + fib(n - 2);
73 }}
74 int main() {{
75     return fib({n});
76 }}
77 """)

```

*Rys 8: Fragment skryptu w języku Python służącego do automatycznego generowania losowych programów w języku C. Przykłady obejmują pętle (for, while), instrukcje warunkowe (switch-case), operacje bitowe oraz funkcje rekurencyjne (factorial, fibonacci). Kod źródłowy wygenerowany w ten sposób stanowił podstawę zbioru uczącego dla modelu językowego.*

### 2.3.2 Kompilacja i ekstrakcja sekcji `_text`

Dla każdego pliku `.c` uruchamiano proces kompilacji za pomocą kompilatora Clang z całkowicie wyłączoną optymalizacją (`-O0`). W celu wyodrębnienia jedynie sekcji `_text` (czyli kodu wykonywalnego), przygotowano skrypt bashowy, który:

1. Kompilował program do pliku wykonywalnego `.out`.
2. Lokalizował offset i rozmiar sekcji `_text` przy użyciu narzędzia `otool`.
3. Wykonywał ekstrakcję odpowiedniego fragmentu binarnego za pomocą `dd`.
4. Konwertował wynik do formatu heksadecymalnego przy użyciu `hexdump`. [22], [23]

Dzięki temu dla każdego programu otrzymywano odpowiadający mu czysty ciąg HEX zapisany w pliku .hex.

```
1  #!/bin/bash
2
3  SOURCE_FILE="$1"
4  OUTDIR="$2"
5
6  BASENAME=$(basename "$SOURCE_FILE" .c)
7  OUT_EXE="${OUTDIR}/${BASENAME}.out"
8  TEXT_BIN="${OUTDIR}/${BASENAME}.bin"
9  TEXT_HEX="${OUTDIR}/${BASENAME}.hex"
10
11 # Kompilacja
12 clang -O0 -o "$OUT_EXE" "$SOURCE_FILE" 2>/dev/null || exit 1
13
14 # Pobierz offset i size z otool
15 SECTION_INFO=$(otool -l "$OUT_EXE" | awk '
16     $1 == "sectname" && $2 == "__text" {in_text=1}
17     in_text && $1 == "offset" {offset=$2}
18     in_text && $1 == "size"    {size=$2}
19     in_text && $1 == "align"  {print offset, size; exit}
20 ')
21
22 OFFSET=$(echo "$SECTION_INFO" | awk '{print $1}')
23 SIZE_HEX=$(echo "$SECTION_INFO" | awk '{print $2}')
24 SIZE_HEX_CLEAN=$(echo "$SIZE_HEX" | sed 's/^0x//')
25 SIZE=$((16#$SIZE_HEX_CLEAN))
26
27
28 # Wyciągnięcie .text
29 dd if="$OUT_EXE" of="$TEXT_BIN" bs=1 skip=$OFFSET count=$SIZE status=none
30
31 # Zamiana binarki na czysty ciąg hexów
32 hexdump -v -e '/1 "%02x"' "$TEXT_BIN" > "$TEXT_HEX"
33
```

Rys 9: Skrypt bashowy wykorzystywany do kompilacji programów w języku C oraz wyodrębniania sekcji `_text` z wygenerowanego pliku wykonywalnego. Skrypt automatycznie lokalizuje offset i rozmiar sekcji `_text` przy użyciu `otool`, a następnie wycina odpowiadający fragment bajtów i konwertuje go do formatu heksadecymalnego za pomocą `hexdump`.

### 2.3.3 Tworzenie par $HEX \rightarrow C$

Po zakończeniu kompilacji i ekstrakcji sekcji `_text`, skrypt Python automatycznie budował rekordy danych w formacie JSON, zawierające:

- „hex” – kod maszynowy jako ciąg znaków heksadecymalnych,
- „source” – oryginalny kod źródłowy w języku C.

Przykładowe rekordy:

```
{
  "source": "int main() { int a = 31, b = 37; return a & b; }",
  "hex": "ff4300d1ff0f00b9e8038052e80b00b9a8048052e80700b9e80b40b9e90740b90001090aff430091c0035fd6"
},
{
  "source": "int main() { int a = 7; if (a != 83) return 1; else return 0; }",
  "hex": "ff4300d1ff0f00b9e8008052e80b00b9e80b40b9084d0171e8179flaa80000370100001428008052e80f00b903000014ff0f00b901000014e00f40b9ff430091c0035fd6"
},
{
  "source": "int main() { int y = 72; if (y < 43) return 1; else return 0; }",
  "hex": "ff4300d1ff0f00b908098052e80b00b9e80b40b908ad0071e8b79flaa80000370100001428008052e80f00b903000014ff0f00b901000014e00f40b9ff430091c0035fd6"
},
}
```

Całość została zapisana w postaci jednej struktury listowej w pliku z rozszerzeniem „.JSON”.

### 2.3.4 Format i struktura danych

Plik wynikowy zawiera standardową strukturę JSON i mógł być wczytywany bezpośrednio do klasy `Dataset` zaimplementowanej w PyTorchu. Dane wejściowe były tokenizowane przy pomocy tokenizera T5 (`T5Tokenizer`), z ograniczeniem do długości 512 tokenów. Padding był używany dla ujednolicenia długości sekwencji, a tokeny paddingu były ignorowane w obliczaniu straty.

## 2.4 Wykorzystanie modelu językowego

W ramach projektu zaimplementowano i przetestowano trzy różne modele sekwencyjne, których zadaniem było automatyczne przekształcanie kodu maszynowego w formacie HEX na odpowiadający mu kod źródłowy w języku C. Wszystkie modele opierają się na architekturze typu `Transformer` i zostały zrealizowane w środowisku Python z wykorzystaniem bibliotek `PyTorch` oraz `Transformers` (Hugging Face). Ich wspólną cechą jest podejście `sequence-to-sequence`, w którym dane wejściowe i wyjściowe są traktowane jako sekwencje tokenów tekstowych [24].

### 2.4.1 Klasyczny Transformer

Pierwszym z testowanych rozwiązań był własnoręcznie zaimplementowany Transformer, oparty bezpośrednio na module „torch.nn.Transformer”. Model ten nie korzystał z gotowego pretrenowania i był trenowany od zera na przygotowanych danych wejściowych HEX i kodzie C.

Kluczowe cechy implementacji:

- Wejście: sekwencje bajtów w postaci tokenów liczbowych (0-255), przekształcane przez warstwę „Embedding”.
- Wyjście: sekwencja znaków ASCII (0-127), generowana krok po kroku przez dekodera.
- Struktura: 6 warstw enkodera i 6 warstw dekodera, 8 głowic uwagi (multi-head attention), rozmiar wektora embeddingu: 512, rozmiar warstwy feed-forward: 768.
- Maskowanie: użyto funkcji `generate_square_subsequent_mask()`, by zapewnić autoregresyjność dekodera (maskowanie przyszłych tokenów).
- Trenowanie: pełny proces optymalizacji realizowany ręcznie: przekazanie src i tgt, obliczanie straty, propagacja wsteczna, `optimizer.step()`.

Model ten stanowił punkt odniesienia dla bardziej zaawansowanych modeli zewnętrznych. Jego przewagą była pełna kontrola nad architekturą, lecz wadą – brak wcześniejszej wiedzy (brak pretreningu), co skutkowało niższą jakością wyników przy porównywalnym czasie trenowania [22].

### 2.4.2 T5-small (pretrenowany Transformer typu encoder-decoder)

Drugim testowanym modelem był T5-small – pretrenowany model językowy z biblioteki Hugging Face, zawierający około 60 milionów parametrów. Został zaprojektowany jako uniwersalne rozwiązanie dla zadań NLP w podejściu text-to-text – przyjmuje tekst jako wejście i generuje tekst jako wyjście, bez względu na charakter zadania.

Parametry i środowisko:

- Tokenizer: T5Tokenizer (bazujący na SentencePiece),
- Max sequence length: 512 tokenów,

- Batch size: 16,
- Optymalizator: AdamW (learning rate: 5e-5),
- Epoki: 3,
- Maskowanie: `labels[labels == pad_token_id] = -100` (zgodnie z wymaganiami CrossEntropyLoss).

Dzięki wcześniejszemu pretrenowaniu na dużych zbiorach tekstu model T5-small był w stanie szybciej dopasować się do konkretnego zadania translacji HEX → C. Pomimo niewielkiego rozmiaru, model osiągał satysfakcjonujące wyniki, szczególnie dla krótszych i prostszych przykładów.

#### 2.4.3 T5-base (rozszerzona wersja T5)

Trzecim testowanym wariantem był T5-base, większy i bardziej pojemny model, zawierający ok. 220 milionów parametrów. Pod względem struktury model ten był identyczny z T5-small, lecz dysponował większą liczbą warstw i parametrów w każdej z nich. Różnice względem T5-small:

- Większy embedding i liczba głowic attention (większa precyzja kontekstowa),
- Wydłużony czas treningu (konieczność użycia GPU o większej pamięci),
- Lepsze wyniki na bardziej złożonych przypadkach (np. funkcje rekurencyjne, kod z `printf()`).

#### 2.4.4 Proces treningu i ewaluacji

Trening każdego modelu odbywał się w identycznym środowisku obliczeniowym (GPU CUDA, PyTorch). Dane wejściowe były przetwarzane za pomocą tokenizera, a następnie przekazywane do modelu w formie `input_ids` i `attention_mask`. Kod źródłowy (target) podlegał tokenizacji jako `labels`. Po każdej epoce zapisywano model do osobnego katalogu, co umożliwiało późniejszą ocenę działania przy różnych wielkościach zbiorów treningowych.

## 2.5 Analiza zbioru danych uczących

Zbiór danych przygotowany zgodnie z procedurą opisaną w punkcie 2.3 został następnie poddany analizie ilościowej i strukturalnej. Celem tej analizy było określenie jakości, różnorodności oraz typowej długości przykładów wykorzystywanych podczas trenowania modeli językowych. Każdy rekord w zbiorze zawierał:

- pole "hex" – zawierające kod maszynowy reprezentowany jako łańcuch bajtów w formacie heksadecymalnym,
- pole "source" – odpowiadający kod źródłowy programu w języku C.

### 2.5.1 Rozmiary i warianty zbiorów

Do eksperymentów przygotowano wiele wersji zbiorów danych o różnej liczności, przedstawionych w tabeli 2, co umożliwiło testowanie wpływu rozmiaru danych uczących na jakość generowanego kodu. Przykładowe zbiory:

*Tabela 2: Przedstawienie poszczególnych zbiorów danych wraz z ich zastosowaniem*

| Nazwa zbioru  | Liczba przykładów | Zastosowanie             |
|---------------|-------------------|--------------------------|
| dataset_1000  | 1 000             | szybki test funkcjonalny |
| dataset_5000  | 5 000             | małoskalowy trening      |
| dataset_10000 | 10 000            | standardowe uczenie      |
| dataset_20000 | 20 000            | eksperymenty z T5-base   |
| dataset_test  | ~50               | ewaluacja końcowa        |

### 2.5.2 Charakterystyka danych

Programy źródłowe zawarte w zbiorze obejmowały różnorodne konstrukcje języka C, takie jak:

- działania arytmetyczne i logiczne,
- instrukcje warunkowe (if, else, switch-case),
- pętle (for, while),
- funkcje (w tym rekurencyjne),

- przykłady z użyciem printf() z różnymi specyfikatorami formatu.

Każdy program był składniowo poprawny i możliwy do skompilowania przy użyciu kompilatora Clang z wyłączoną optymalizacją (-O0).

### *2.5.3 Długość sekwencji i tokenizacja*

Przeprowadzona analiza długości danych wykazała, że średnia długość wejściowego ciągu HEX (przed tokenizacją) wynosiła od 150 do 300 bajtów, natomiast odpowiadający kod źródłowy w języku C mieścił się zazwyczaj w zakresie 100–200 znaków. Po zastosowaniu tokenizacji z użyciem modelu T5 (SentencePiece), całkowita długość sekwencji — obejmująca zarówno wejście, jak i wyjście — nie przekraczała 512 tokenów. Wszystkie sekwencje zostały ujednolicone poprzez stosowanie paddingu lub ucinania, w celu zapewnienia zgodności z wymaganym formatem wejściowym modeli językowych. Zastosowany tokenizer (T5Tokenizer) bazował na SentencePiece i zapewniał zgodność z wymaganiami architektury T5. Dzięki temu możliwe było trenowanie modeli z zachowaniem spójnego rozmiaru wejść i wyjść.

## **2.6 Przetwarzanie danych wejściowych**

Zanim dane mogły zostać przekazane do modelu językowego, konieczne było ich odpowiednie przygotowanie w formie zgodnej z wymaganiami architektury Transformer. Proces ten obejmował tokenizację tekstu, tworzenie masek uwagi oraz zarządzanie długością sekwencji. Zastosowano podejście zgodne z biblioteką Hugging Face Transformers, z wykorzystaniem gotowych metod dostępnych w klasie T5Tokenizer.

### *2.6.1 Tokenizacja*

Tokenizacja polegała na zamianie tekstu (zarówno wejściowego HEX, jak i wyjściowego kodu C) na sekwencję tokenów liczbowych (ID). Każdy ciąg znaków był przetwarzany za pomocą metody tokenizer(...), z ustaloną maksymalną długością sekwencji równą 512 tokenów. Dłuższe fragmenty były automatycznie przycinane, natomiast krótsze dopełniane tokenem <pad>. Tokeny wyjściowe (labels) – czyli kod C – były również tokenizowane w ten sam sposób. W celu poprawnego działania funkcji straty CrossEntropyLoss, wszystkie tokeny <pad> w etykietach zostały zamienione na wartość -100, która jest domyślnie ignorowana przez mechanizm obliczania błędu [22].

### 2.6.2 Attention mask i padding

Dla każdej sekwencji wejściowej generowano dodatkową maskę uwagi (attention\_mask), która przyjmuje wartość 1 dla tokenów rzeczywistych i 0 dla tokenów paddingu. Dzięki temu model był w stanie skupić się tylko na faktycznej treści, ignorując wypełnienie. Zastosowanie paddingu umożliwiała tworzenie batchy o jednakowych wymiarach, co było wymagane przy trenowaniu modeli w trybie równoległym (np. na GPU). Padding był dodawany automatycznie do obu sekwencji: wejściowej (input\_ids) oraz wyjściowej (labels) [22].

### 2.6.3 Format danych przekazywanych do modelu

Po przygotowaniu, każda próbka danych była reprezentowana jako słownik zawierający trzy główne elementy:

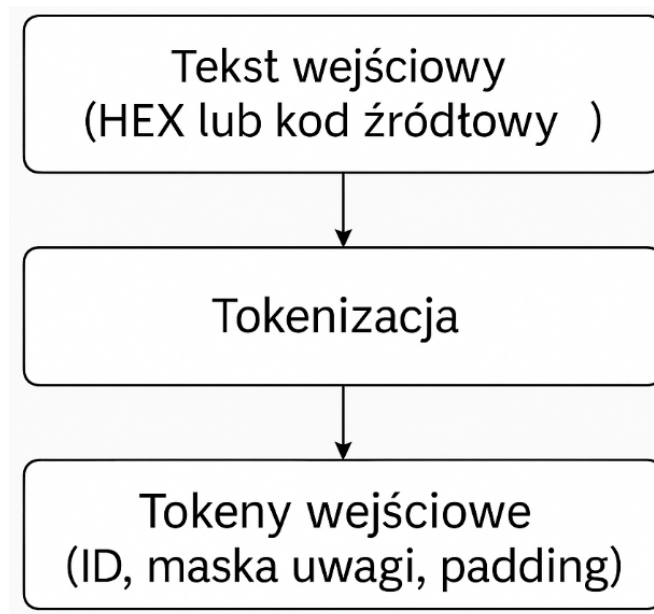
- input\_ids: zakodowana sekwencja wejściowa (HEX),
- attention\_mask: maska informująca model, które tokeny są istotne,
- labels: zakodowana sekwencja kodu C (z -100 dla paddingu).

Przykładowy batch danych wyglądał następująco (uproszczony):

```
{
  "input_ids":      [106, 243, 88, 9, 4, 5, ..., 0, 0],
  "attention_mask": [ 1,  1,  1, 1, 1, 1, ..., 0, 0],
  "labels":         [ 71, 181, 15, 6, 78, ..., -100, -100]
}
```

Rys 10: Przykład przygotowanego batcha danych przekazywanego do modelu językowego.





*Rys 11: Schemat przetwarzania danych wejściowych przed przekazaniem do modelu językowego. Tekst w postaci kodu maszynowego (HEX) lub źródłowego (C) podlega tokenizacji, w wyniku której uzyskiwane są sekwencje wejściowe zawierające identyfikatory tokenów (`input_ids`), maskę uwagi (`attention_mask`) oraz padding.*

### 3. EKSPERYMENTY I WYNIKI.

#### 3.1 Konfiguracja eksperymentów i środowiska testowego

W celu weryfikacji skuteczności działania zaproponowanego rozwiązania dekompiletora wspomaganego sztuczną inteligencją przeprowadzono szereg eksperymentów porównawczych. Ich celem było zbadanie, jak dobrze różne modele językowe radzą sobie z zadaniem translacji kodu maszynowego (HEX) na odpowiadający mu kod źródłowy w języku C. Eksperymenty przeprowadzono w kontrolowanym środowisku obliczeniowym, z zastosowaniem ujednoliconej procedury treningu i testowania, aby umożliwić rzetelne porównanie wyników. W badaniach uwzględniono trzy różne modele. Pierwszym z nich był klasyczny Transformer, zaimplementowany ręcznie z wykorzystaniem `torch.nn.Transformer`, bez korzystania z pretrenowanych wag. Model ten reprezentował podejście bazowe – w pełni własne, trenowane od zera. Pozostałe dwa modele, T5-small oraz T5-base, zostały zaimportowane z biblioteki „Hugging Face Transformers” i poddane procesowi dostosowania (fine-tuning) na danych specyficznych dla zadania dekompilacji. Model T5-small zawiera około 60 milionów parametrów, natomiast T5-base – ponad 220 milionów. Zastosowanie obu wariantów pozwoliło na ocenę wpływu rozmiaru modelu na jakość generowanego kodu oraz na wykorzystanie zasobów podczas treningu.

Dane treningowe wykorzystane w eksperymentach zostały przygotowane w oparciu o wcześniej opisany pipeline (rozdział 2.3), obejmujący automatyczne generowanie losowych programów w języku C, ich kompilację, ekstrakcję sekcji `_text`, konwersję do formatu HEX oraz zapis w formacie JSON. Każdy rekord w zbiorze danych zawierał parę "hex" – reprezentującą instrukcje maszynowe – oraz "source" – zawierającą kod źródłowy programu. Zbiory te zostały przygotowane w wersjach o różnej liczebności: 1000, 5000, 10000, 20000, 50000 i 100000 przykładów. W każdej iteracji eksperymentu model trenowano na jednym z tych zestawów, natomiast ocena jakości działania odbywała się na osobnym zbiorze `dataset_test`, zawierającym około 50 niezależnych przykładów. Środowisko eksperymentalne oparto na języku Python w wersji 3.10, z wykorzystaniem bibliotek PyTorch (w wersji 2.x) oraz Transformers. Modele T5 trenowano z użyciem wbudowanego tokenizera `T5Tokenizer`, który wykorzystuje algorytm `SentencePiece`. Dane wejściowe i wyjściowe podlegały tokenizacji, a następnie były konwertowane na tensory: `input_ids`, `attention_mask` i `labels`. Maksymalna długość sekwencji została ograniczona do 512 tokenów. Tokeny `<pad>` były automatycznie maskowane w etykietach wartością `-100`, co pozwalało funkcji straty `CrossEntropyLoss` ignorować padding podczas obliczania gradientów.

Tak skonfigurowane środowisko eksperymentalne pozwoliło na systematyczne porównanie jakości generowanego kodu oraz ocenę wpływu liczby przykładów treningowych i złożoności modelu na końcowe rezultaty dekompilacji.

### 3.1.1 Trenowanie modelu Transformer

Model Transformer został zaimplementowany w języku Python z wykorzystaniem biblioteki PyTorch. W odróżnieniu od modeli pretrenowanych, konstrukcja tego modelu została przygotowana od podstaw, z pełną kontrolą nad wszystkimi komponentami: embeddingiem, kodowaniem pozycyjnym, enkoderem, dekodere oraz warstwą wyjściową. Dzięki temu możliwe było dokładne dostosowanie struktury modelu do zadania sekwencyjnej transformacji kodu maszynowego (HEX) na kod źródłowy w języku C. Za bazę przyjęto standardową architekturę `torch.nn.Transformer`, dostępną od wersji 1.8 PyTorch z obsługą trybu `batch_first=True`. Konfiguracja obejmowała 3 warstwy enkodera i 3 warstwy dekodera, 4 głowy uwagi, rozmiar embeddingu 256 oraz wymiar warstwy feed-forward równy 2048. Wejście i wyjście obsługiwane były przez jedną warstwę embeddingu wspólną dla enkodera i dekodera. Model wyposażono w klasyczną sinusoidalną warstwę kodowania pozycji (`PositionalEncoding`), której celem było umożliwienie modelowi rozróżnienia pozycji tokenów w sekwencji, ponieważ architektura Transformera nie posiada mechanizmu czasu ani rekurencji.

**Listing 3.1:** Funkcja generująca maskę dekodera w modelu Transformer

```
class PositionalEncoding(nn.Module):  
    def __init__(self, d_model, max_len=512):  
        ...
```

Sekwencje wejściowe i wyjściowe były reprezentowane jako tokeny tekstowe, przetworzone za pomocą tokenizera `T5Tokenizer` (`SentencePiece`). Umożliwiło to bezproblemowe mapowanie bajtów HEX i znaków C na wspólną przestrzeń słownikową. Dzięki użyciu tokenizera z rodziny T5 zachowano spójność reprezentacji tokenów pomiędzy modelem Transformer a modelami T5-small i T5-base, co ułatwiało porównanie wyników.

Dla każdego przykładu (HEX  $\rightarrow$  C) dane wejściowe i wyjściowe były tokenizowane z maksymalną długością 512 tokenów. Tokeny `<pad>` były używane do uzupełnienia sekwencji do pełnej długości, a ich indeks (`tokenizer.pad_token_id`) był wykorzystywany do maskowania w dekodrze oraz ignorowania w funkcji straty (`ignore_index=-100`). Aby umożliwić modelowi autoregresję (przewidywanie tokenu na podstawie wcześniejszych), zastosowano technikę teacher forcing: sekwencja docelowa (`target_ids`) była przesuwana o

jeden token – wejście do dekodera (`tgt_input`) zawierało wszystkie tokeny poza ostatnim, a etykieta (`tgt_label`) zawierała te same tokeny, przesunięte w lewo, bez pierwszego.

**Listing 3.2:** Funkcja generująca maskę dekodera

```
tgt_input = tgt[:, :-1]
tgt_label = tgt[:, 1:]
```

Aby dekodер nie mógł podglądać przyszłych tokenów, zastosowano tzw. `subsequent mask`, generowaną dynamicznie przy każdej iteracji batcha:

**Listing 3.3:** Przygotowanie danych wejściowych (tokenizacja i przesunięcie targetu)

```
def generate_square_subsequent_mask(sz):
    return torch.triu(torch.full((sz, sz), float('-inf')), diagonal=1)
```

Maska ta była przekazywana do dekodera jako `tgt_mask`, a tokeny paddingu były maskowane osobno jako `src_key_padding_mask` i `tgt_key_padding_mask`. Funkcją kosztu była klasyczna `CrossEntropyLoss`, skonfigurowana z parametrem `ignore_index=pad_token_id`, dzięki czemu padding nie wpływał na wynik. Wynik modelu (logits) miał postać `[batch, seq_len, vocab_size]`, dlatego był przekształcany do `[batch × seq_len, vocab_size]` przed obliczeniem straty.

Model był trenowany przez 3 epoki z wykorzystaniem optymalizatora Adam (`lr = 5e-4`). Poniżej uproszczona wersja kodu pętli:

**Listing 3.4:** Główna pętla treningowa modelu Transformer

```
for epoch in range(epochs):
    model.train()
    for batch in dataloader:
        src = batch["input_ids"].to(device)
        tgt = batch["target_ids"].to(device)

        tgt_input = tgt[:, :-1]
        tgt_label = tgt[:, 1:]

        tgt_mask =
generate_square_subsequent_mask(tgt_input.size(1)).to(device)
        src_pad_mask = (src == tokenizer.pad_token_id)
        tgt_pad_mask = (tgt_input == tokenizer.pad_token_id)
```

```
logits = model(src, tgt_input, src_pad_mask, tgt_pad_mask, tgt_mask)
logits = logits.reshape(-1, logits.size(-1))
tgt_label = tgt_label.reshape(-1)

loss = criterion(logits, tgt_label)
loss.backward()
optimizer.step()
optimizer.zero_grad()
```

Parametry treningu:

- Liczba epok: 3
- Batch size: 16
- Max sequence length: 512
- Optimizer: Adam (lr = 5e-4)
- Tokenizacja: T5Tokenizer (SentencePiece)
- Ignore index: -100
- Model output: .pth (checkointy wag PyTorch)

Po zakończeniu treningu model był zapisywany do pliku, a czas trwania treningu był logowany i został zaprezentowany w podrozdziale 3.3.

### 3.1.2 Trenowanie modelu T5-small

Model T5-small został wykorzystany jako architektura bazowa do realizacji zadania dekompilacji kodu maszynowego do kodu źródłowego C. Jest to model typu encoder–decoder, zawierający około 60 milionów parametrów. Dzięki uniwersalnej architekturze „text-to-text” i dostępności w bibliotece Hugging Face Transformers, możliwe było szybkie dostosowanie go do konkretnego zadania za pomocą procesu fine-tuningu.

Dane były ładowane do modelu za pomocą klasy `CodeHexDataset`, która implementuje interfejs `torch.utils.data.Dataset`. Wewnątrz metody `__getitem__`, oba pola były tokenizowane przy użyciu `T5Tokenizer`, z maksymalną długością 512 tokenów. Tokeny paddingu były automatycznie maskowane wartością -100, zgodnie z wymaganiami funkcji straty `CrossEntropyLoss`.

**Listing 3.5:** Klasa `Dataset` do tokenizacji danych wejściowych i wyjściowych

```
class CodeHexDataset(Dataset):
    def __init__(self, file_path, tokenizer, max_length=512):
        with open(file_path, "r") as f:
            data = json.load(f)
            self.source = [item["hex"] for item in data]
            self.target = [item["source"] for item in data]
            self.tokenizer = tokenizer
            self.max_length = max_length

    def __getitem__(self, idx):
        source_encoding = self.tokenizer(
            self.source[idx],
            max_length=self.max_length,
            padding="max_length",
            truncation=True,
            return_tensors="pt",
        )
        target_encoding = self.tokenizer(
            self.target[idx],
            max_length=self.max_length,
            padding="max_length",
            truncation=True,
            return_tensors="pt",
        )
        labels = target_encoding["input_ids"]
        labels[labels == self.tokenizer.pad_token_id] = -100
        return {
```

```

        "input_ids": source_encoding["input_ids"].squeeze(0),
        "attention_mask": source_encoding["attention_mask"].squeeze(0),
        "labels": labels.squeeze(0),
    }

```

Model `T5ForConditionalGeneration` oraz tokenizer `T5Tokenizer` zostały załadowane bezpośrednio z repozytorium Hugging Face. W tym przypadku użyto wariantu `"t5-small"`.

**Listing 3.6:** Inicjalizacja modelu i tokenizera

```

model_name = "t5-small"
tokenizer = T5Tokenizer.from_pretrained(model_name)
model = T5ForConditionalGeneration.from_pretrained(model_name)

```

Dane treningowe zostały załadowane do obiektu klasy `DataLoader` z batch size ustawionym na 16.

Trening odbywał się przez 3 epoki. W każdej iteracji wykonywano:

- Forward pass (`model(...)`) z wejściami `input_ids`, `attention_mask`, `labels`,
- Obliczenie funkcji straty (`loss`) – zintegrowanej z architekturą modelu,
- Propagację wsteczną (`loss.backward()`),
- Aktualizację wag (`optimizer.step()`),
- Zerowanie gradientów (`optimizer.zero_grad()`).

**Listing 3.7:** Pętla treningowa modelu T5-small

```

optimizer = torch.optim.AdamW(model.parameters(), lr=5e-5)
device = "cuda" if torch.cuda.is_available() else "cpu"
model.to(device)

for epoch in range(3):
    model.train()

```

```

for batch in dataloader:
    input_ids = batch["input_ids"].to(device)
    attention_mask = batch["attention_mask"].to(device)
    labels = batch["labels"].to(device)

    outputs = model(
        input_ids=input_ids,
        attention_mask=attention_mask,
        labels=labels
    )
    loss = outputs.loss
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

```

Dzięki temu, że funkcja `T5ForConditionalGeneration` automatycznie zajmuje się obliczaniem maskowania i przesunięcia etykiet, kod treningowy pozostaje uproszczony w porównaniu do klasycznego `Transformer`a.

Po zakończeniu treningu model oraz tokenizer zostały zapisane w formacie Hugging Face, co umożliwi ich późniejsze wykorzystanie do generowania kodu źródłowego lub dalszego trenowania.

**Listing 3.8:** Zapis wytrenowanego modelu i tokenizera

```

model.save_pretrained("code_decompiler_model_5000")
tokenizer.save_pretrained("code_decompiler_model_5000")

```

Parametry treningu:

- Model: T5-small (60M parametrów)
- Liczba epok: 3
- Batch size: 16
- Maks. Długość sekwencji: 512 tokenów



- Optymalizator: AdamW
- Learning rate: 5e-5
- Tokenizer: T5Tokenizer (SentencePiece)
- Loss ignore index: -100

### 3.1.3 Trenowanie modelu T5-base

Model T5-base jest rozszerzoną wersją architektury T5-small, wyposażoną w większą liczbę warstw oraz parametrów. Całkowita liczba parametrów tego modelu wynosi około 220 milionów, co pozwala na lepsze uchwycenie złożonych relacji między tokenami, kosztem większego zapotrzebowania na zasoby obliczeniowe. Model został zaimportowany z biblioteki Hugging Face Transformers jako `T5ForConditionalGeneration`, a do tokenizacji zastosowano `T5Tokenizer` zgodny z `SentencePiece`. Tokeny wejściowe (`input_ids`) i wyjściowe (`labels`) zostały ujednolicone do długości 512, a maskowanie paddingu realizowano poprzez przypisanie wartości -100 w etykietach, zgodnie z wymaganiami funkcji straty `CrossEntropyLoss`. W eksperymencie model był trenowany na zestawie danych zawierającym 1000 par wejściowych i wyjściowych. Pomimo niewielkiego rozmiaru zbioru, model ten wymagał mniejszego batch size'u z powodu większego zużycia pamięci GPU. Ustawiono wartość `batch_size = 8`.

#### **Listing 3.9:** Inicjalizacja modelu i przygotowanie danych

```
model_name = "t5-base"
tokenizer = T5Tokenizer.from_pretrained(model_name)
model = T5ForConditionalGeneration.from_pretrained(model_name)

dataset = CodeHexDataset(file_path="dataset_1000.json", tokenizer=tokenizer,
max_length=512)
dataloader = DataLoader(dataset, batch_size=8, shuffle=True)
```

Model został przeniesiony na urządzenie GPU (jeśli dostępne), a do optymalizacji zastosowano AdamW z domyślnym współczynnikiem uczenia `lr = 5e-5`. Trening trwał przez 3 epoki.

**Listing 3.10:** Pętla treningowa dla modelu T5-base

```
for epoch in range(epochs):
    model.train()
    for batch in dataloader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
```

Architektura modelu wewnętrznie obsługuje tworzenie masek dekodera i przesuwanie etykiet (teacher forcing), dlatego kod treningowy jest bardziej kompaktowy i odporny na błędy. Mimo mniejszej liczby przykładów model zdołał nauczyć się podstawowych struktur i operatorów języka C, jednak z uwagi na większą złożoność architektury wymagałby dłuższego treningu i większego zbioru do osiągnięcia pełnego potencjału.

Parametry treningu:

- Model: T5-base (220M parametrów)
- Liczba epok: 3
- Batch size: 8
- Maks. Długość sekwencji: 512 tokenów
- Optymalizator: AdamW

- Learning rate: 5e-5
- Tokenizer: T5Tokenizer
- Pad token ignore: -100

**Listing 3.11:** Zapis modelu T5-base i tokenizera

```
model.save_pretrained("code_decompiler_model_base_1000")
tokenizer.save_pretrained("code_decompiler_model_base_1000")
```

Model ten został zapisany w formacie Hugging Face, co umożliwia jego ponowne wczytanie i wykorzystanie w procesie testowania lub dalszego trenowania na większym zbiorze.

### 3.2 Metody oceny jakości dekompilacji

W celu dokładnej i wieloaspektowej oceny skuteczności modeli językowych w zadaniu dekompilacji kodu maszynowego (w postaci heksadecymalnej) do kodu źródłowego w języku C, zastosowano trzy niezależne i uzupełniające się metryki:

1. Similarity score – dopasowanie ciągów znaków (edytorskie podobieństwo),
2. BLEU score – zgodność sekwencji tokenów (n-gramów),
3. Semantic similarity – ocena zgodności logicznej (funkcjonalnej) kodu.

Każda z nich została dobrana w taki sposób, aby uchwycić inny wymiar poprawności generowanego kodu: od identyczności tekstowej, przez strukturę składniową, aż po spójność semantyczną.

#### 3.2.1 Similarity score – dopasowanie ciągów znaków

Similarity score to prosta, ale bardzo efektywna metryka obliczana za pomocą funkcji SequenceMatcher z biblioteki difflib w Pythonie. Mierzy ona procentową zgodność dwóch ciągów znaków, bazując na długości i kolejności ich wspólnych podciągów (ang. *longest contiguous matching blocks*). W kontekście dekompilacji metryka ta służyła jako podstawowy wskaźnik „literalnej” zgodności wygenerowanego kodu z oryginalnym – uwzględniając wszystkie znaki, spacje, tabulatory, nawiasy, średniki i nazwy zmiennych. Zaletą tej metody jest: bardzo szybka implementacja i interpretacja oraz wrażliwość na dokładność wygenerowanego tekstu

(czy model wygenerował dokładnie to, co trzeba). Natomiast wadą są: minimalna różnica, np. brak średnika, obniża wynik oraz nie rozróżnianie znaczenia (zmiana „x” na „y” traktowana jest jako błąd, nawet jeśli sens pozostaje ten sam).

### 3.2.2 BLEU score – zgodność tokenów

BLEU (ang. *Bilingual Evaluation Understudy*) to klasyczna metryka wykorzystywana w tłumaczeniu maszynowym. Oblicza stopień pokrycia n-gramów (ciągów 1, 2, 3, 4 słów lub tokenów) wygenerowanego tekstu względem tekstu referencyjnego. W testach użyto wersji `sentence_bleu` z pakietu NLTK oraz technik wygładzających (np. `method4` z `SmoothingFunction`), co umożliwiało sensowną ocenę nawet dla bardzo krótkich fragmentów kodu. BLEU był szczególnie użyteczny jako metryka składniowej i lokalnej poprawności wygenerowanego kodu. Wysoki wynik BLEU oznaczał, że struktura składniowa, kolejność tokenów i lokalna zgodność była zbliżona do referencyjnej. Zaletami tej metody są: odporność na różnice kosmetyczne (np. zmiana kolejności wyrażeń), oraz lepsze odzwierciedlenie zgodności składniowej w porównaniu do Similarity score. Aczkolwiek może nadawać wysoki wynik kodowi składniowo poprawnemu, ale logicznie błędnemu (np. błędne dane wejściowe do funkcji `return`).

### 3.2.3 Semantic similarity – zgodność logiczna kodu

Najbardziej zaawansowaną i dostosowaną do specyfiki dekompilacji metryką była ocena semantyczna, mająca na celu sprawdzenie, czy wygenerowany kod zachowuje logikę i funkcjonalność oryginału, nawet jeśli różni się pod względem nazw zmiennych, formatowania, czy struktury nawiasów. W tym celu opracowano specjalną procedurę normalizacji kodu, obejmującą:

- usunięcie znaków formatowania: `{}`, `;`, `\t`, `\n`, itp.,
- zamianę wszystkich nazw identyfikatorów (zmiennych i funkcji) na symbol `X`,
- zachowanie słów kluczowych języka C (`int`, `return`, `if`, `case`, itp.),
- zachowanie wartości liczbowych (np. 2, 4, 10, 0), które są kluczowe dla logiki programu.

Dodatkowo, jeśli zestaw wartości liczbowych w kodzie wygenerowanym nie pokrywał się z oryginalnym, wynik końcowy był obniżany o 15%, co miało na celu ukaranie logicznych przekłamań — np. jeśli `return power(2, 4)` zamieniono na `return power(5, 3)`, choć składnia była poprawna. Zaletami tej metody są: lepsze odzwierciedlenie ludzkiego postrzegania sensu kodu, uwzględnia semantykę i wartości, odporność na różnice składniowe. Natomiast nadal nie gwarantuje pełnej zgodności funkcjonalnej (nie uruchamia kodu), oraz nie wykrywa subtelnych błędów semantycznych (np. błędna zmienna, ale ta sama wartość).

### **3.2.4 Uzasadnienie wyboru metryk**

Wybór powyższych trzech metryk był podyktowany koniecznością uwzględnienia różnych poziomów poprawności kodu:

- Similarity – szybka ocena tekstowa,
- BLEU – ocena zgodności sekwencji,
- Semantic – ocena logicznego zachowania programu.

Dzięki temu możliwe było stworzenie pełnego profilu jakości dekompilacji: od zgodności na poziomie znaków po ocenę „czy człowiek byłby w stanie zrozumieć i wykorzystać wygenerowany kod”.

### **3.3 Wyniki dla poszczególnych modeli**

Aby ocenić skuteczność różnych architektur modeli językowych w zadaniu dekompilacji kodu maszynowego do języka C, przeprowadzono testy porównawcze trzech modeli: własnej implementacji Transformera, modelu T5-small oraz T5-base. Każdy z modeli został przetrenowany na zestawie danych o różnych rozmiarach: 1000, 5000, 10000 oraz 20000 przykładów, co umożliwiło zaobserwowanie wpływu rozmiaru zbioru uczącego na jakość generowanego kodu. Wyniki zostały ocenione trzema metrykami:

- Similarity score – dopasowanie tekstu (SequenceMatcher),
- BLEU score – zgodność n-gramów (sentence\_bleu),
- Semantic similarity – porównanie funkcjonalnej zgodności kodu (z uwzględnieniem wartości liczbowych i normalizacji nazw).

### 3.3.1 Przebieg treningu modelu Transformer

Tabela 3: Trenowanie modelu Transformer:

| Model       | Dataset | Liczba epok | Czas treningu [sec/min] | Funkcja Loss (epoka 1) | Funkcja Loss (epoka 2) | Funkcja Loss (epoka 3) |
|-------------|---------|-------------|-------------------------|------------------------|------------------------|------------------------|
| Transformer | 1000    | 3           | 19,08/0,32              | 0,90                   | 0,44                   | 0,21                   |
| Transformer | 5000    | 3           | 94,21/1,57              | 0,15                   | 0,20                   | 0,19                   |
| Transformer | 10000   | 3           | 188,38/3,14             | 0,13                   | 0,15                   | 0,11                   |
| Transformer | 20000   | 3           | 379,08/6,32             | 0,12                   | 0,11                   | 0,17                   |

Tabela 2 przedstawia szczegółowe parametry trenowania własnej implementacji modelu Transformer w zależności od rozmiaru zbioru uczącego (1000, 5000, 10000 oraz 20000 przykładów). Każdy z eksperymentów obejmował trzy epoki uczenia, a dane zebrano na podstawie treningów przeprowadzonych na pojedynczym GPU. Dla najmniejszego zbioru danych, obejmującego 1000 przykładów, całkowity czas treningu wyniósł 19,08 sekundy (około 0,32 minuty), a funkcja straty (loss) spadła z początkowej wartości 0,90 w pierwszej epoce do zaledwie 0,21 w epoce trzeciej. Oznacza to bardzo szybkie dopasowanie modelu do danych — przy czym należy zaznaczyć, że tak mały zbiór sprzyja łatwemu „nauczeniu się” rozkładu wejścia, ale może prowadzić do przeuczenia. W przypadku zbioru o wielkości 5000 przykładów czas treningu wzrósł do 94,21 sekundy (1,57 minuty), a wartości funkcji straty kształtowały się odpowiednio: 0,15 (epoka 1), 0,20 (epoka 2), 0,19 (epoka 3). W tym przypadku widać nietypowy wzrost wartości straty w drugiej epoce, który może być efektem oscylacji podczas optymalizacji lub fluktuacji wynikających z losowej inicjalizacji batch. Dla zbioru 10000 czas treningu wyniósł 188,38 sekundy (3,14 minuty), a wartości loss w kolejnych epokach spadły z 0,13 do 0,11. Trend ten wskazuje na wyraźne ustabilizowanie procesu uczenia oraz skuteczne dopasowanie modelu do coraz większego zbioru. Największy zbiór, obejmujący 20000 przykładów, trenowano przez 379,08 sekundy (około 6,32 minuty). Tu obserwujemy nieco nietypowy przebieg wartości funkcji straty: 0,12 (ep. 1), 0,11 (ep. 2), ale wzrost do 0,17 w epoce 3. Może to sugerować efekt tzw. overfittingu lokalnego, lub wskazywać na potrzebę zastosowania innej strategii regularyzacji (np. zmniejszenia learning rate lub zastosowania dropoutu). Analiza tabeli potwierdza, że wraz ze wzrostem rozmiaru zbioru dane stają się bardziej reprezentatywne, co skutkuje mniejszymi wartościami funkcji straty. Jednocześnie większe zbiory wymagają dłuższego czasu trenowania, jednak proporcjonalnie zwiększają zdolność modelu do generalizacji. Transformer jako własna implementacja wykazał stabilne właściwości uczenia i dobrze skalował się względem liczby przykładów.

### 3.3.2 Przebieg treningu modelu T5-small

Tabela 4: Trenowanie modelu T5-small:

| Model    | Dataset | Liczba epok | Czas treningu [sec/min] | Funkcja Loss (epoka 1) | Funkcja Loss (epoka 2) | Funkcja Loss (epoka 3) |
|----------|---------|-------------|-------------------------|------------------------|------------------------|------------------------|
| T5-small | 1000    | 3           | 330,13/5,50             | 2,16                   | 1,58                   | 0,93                   |
| T5-small | 5000    | 3           | 1293,65/21,56           | 0,61                   | 0,34                   | 0,31                   |
| T5-small | 10000   | 3           | 2747,77/45,80           | 0,34                   | 0,23                   | 0,20                   |
| T5-small | 20000   | 3           | 5113,07/85,22           | 0,22                   | 0,20                   | 0,16                   |

Tabela 3 przedstawia szczegółowe wyniki treningu modelu T5-small na czterech zbiorach danych o różnej liczebności: 1000, 5000, 10000 oraz 20000 par kodu maszynowego i odpowiadającego mu kodu źródłowego. Każdy z eksperymentów obejmował trzy epoki uczenia. Dane zebrano podczas treningów przeprowadzanych na tym samym sprzęcie co w przypadku modelu Transformer, co pozwala na bezpośrednie porównanie obu architektur. W przypadku najmniejszego zbioru danych (1000 przykładów), czas treningu wyniósł 330,13 sekundy, czyli około 5,5 minuty. Wartości funkcji straty spadały stabilnie w każdej kolejnej epoce: od 2,16 w pierwszej, przez 1,58 w drugiej, aż do 0,93 w trzeciej epoce. To sugeruje, że model szybko dopasował się do danych, co jest efektem zarówno skutecznego mechanizmu uwagi, jak i wcześniejszego pretrenowania modelu T5-small na dużych zbiorach danych programistycznych. Wraz ze wzrostem liczby przykładów w zbiorze, czas treningu ulegał proporcjonalnemu wydłużeniu. Dla zbioru 5000 trening trwał 1293,65 sekundy (ok. 21,56 minut), a funkcja straty obniżyła się z 0,61 do 0,31. Widać tu bardzo szybkie i stabilne dopasowanie modelu, bez oznak przeuczenia. Dla zbioru 10000 przykładów czas treningu wyniósł 2747,77 sekundy (45,8 minuty), a wartości funkcji straty osiągnęły odpowiednio: 0,34, 0,23 oraz 0,20, co wskazuje na kontynuację poprawy jakości predykcji przy większej liczbie danych. Największy zbiór danych – 20000 par – trenowany był przez ponad 85 minut (5113,07 sekundy). Spadek wartości funkcji straty był nadal zauważalny: z 0,22 na początku do 0,16 po trzeciej epoce, przy czym dynamika poprawy była już mniejsza. Świadczy to o tym, że model osiągnął punkt, w którym kolejne zwiększanie danych prowadzi do poprawy jakości, ale z malejącymi przyrostami efektywności. Wnioski z tabeli potwierdzają dobrą skalowalność modelu T5-small oraz skuteczność jego pretrenowanej architektury. Model radził sobie z rosnącą liczbą danych bardzo dobrze, osiągając niskie wartości straty już po kilku epokach. Jednocześnie, w porównaniu do Transformera, czas treningu był znacznie dłuższy, co jest ceną za większą złożoność architektury i liczby parametrów.

### 3.3.3 Przebieg treningu modelu T5-base

Model T5-base został przetestowany w ograniczonym zakresie, wyłącznie na zbiorze danych obejmującym 1000 par treningowych (HEX + kod źródłowy w języku C). Powodem tego była bardzo duża liczba parametrów sieci (około 220 milionów), co wiąże się z wysokimi wymaganiami obliczeniowymi oraz pamięciowymi. Trening przeprowadzono na pojedynczym GPU klasy konsumenckiej, co w przypadku tak złożonego modelu znacząco wpłynęło na czas jego wykonywania. Cały proces uczenia trwał około 62,5 minuty, czyli 3750,76 sekundy, i obejmował trzy epoki. Wartość funkcji straty systematycznie malała w kolejnych epokach: w pierwszej wyniosła 0,6714, w drugiej 0,5016, a w trzeciej 0,3433. Obserwowana tendencja spadku loss wskazuje na to, że nawet przy ograniczonym zbiorze danych model uczył się w sposób skuteczny, co może być wynikiem jego wcześniejszego pretrenowania na dużych korpusach kodu źródłowego. Z uwagi na bardzo długi czas trenowania i ograniczenia sprzętowe, zrezygnowano z dalszych eksperymentów z T5-base na większych zbiorach danych (5000, 10000, 20000). W dalszej części pracy model ten pełni funkcję punktu odniesienia pod względem jakości, ale nie bierze udziału w pełnym porównaniu eksperymentalnym ze względu na brak możliwości skalowania treningu.

### 3.3.4 Analiza porównawcza

Tabela 5: Przedstawienie porównawcze wyników trzech modeli:

| Model       | Rozmiar zbioru | Similarity [%] | BLEU [%] | Semantic [%] |
|-------------|----------------|----------------|----------|--------------|
| Transformer | 1000           | 42,39          | 3,58     | 42,74        |
| Transformer | 5000           | 48,13          | 8,95     | 45,35        |
| Transformer | 10000          | 65,83          | 17,69    | 59,30        |
| Transformer | 20000          | 43,87          | 5,30     | 45,19        |
| T5-small    | 1000           | 63,67          | 6,29     | 63,71        |
| T5-small    | 5000           | 83,38          | 14,19    | 81,46        |
| T5-small    | 10000          | 83,60          | 14,43    | 81,90        |
| T5-small    | 20000          | 85,64          | 14,41    | 86,11        |
| T5-base     | 1000           | 76,87          | 12,09    | 74,34        |

Tabela 4 przedstawia wyniki uzyskane przez trzy różne modele dekompileatorów: własną implementację modelu Transformer, model T5-small oraz T5-base. Modele oceniano na czterech rozmiarach zbioru danych (1000, 5000, 10000, 20000 przykładów), mierząc jakość ich predykcji przy użyciu trzech metryk: Similarity (dopasowanie znaków), BLEU (n-gramowe



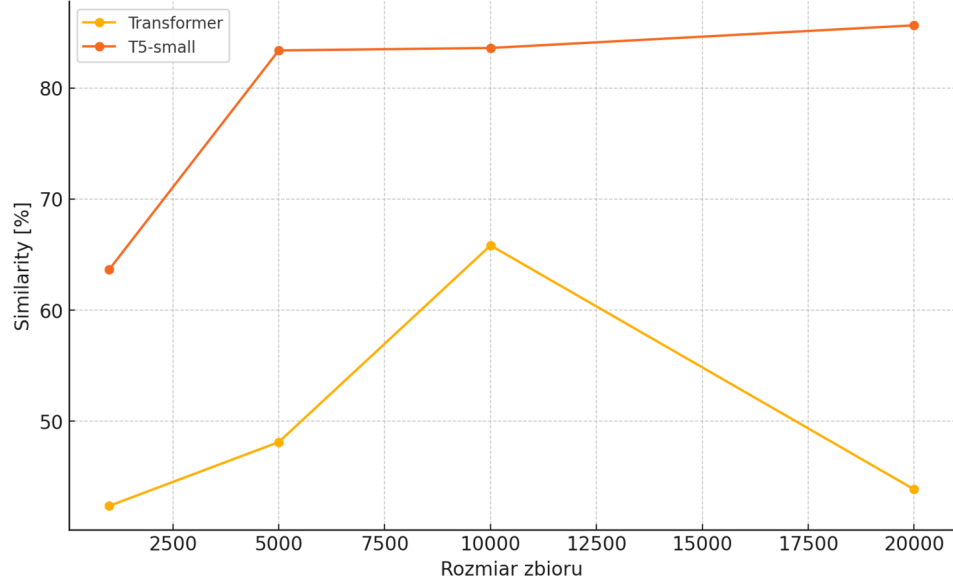
podobieństwo) oraz Semantic Similarity (zgodność funkcjonalna). W przypadku własnego modelu Transformer obserwowano stopniową poprawę wyników w miarę zwiększania rozmiaru zbioru danych – osiągając najlepsze rezultaty dla zbioru 10000: Similarity na poziomie 65,83%, BLEU 17,69% i Semantic 59,30%. Co ciekawe, przy zbiorze 20000 nastąpił wyraźny spadek wyników we wszystkich trzech metrykach. Taki efekt może wskazywać na problem z przeuczeniem, niedostosowaniem architektury do większej ilości danych lub zbyt dużym rozproszeniem jakości danych w największym zbiorze. Zaskakująco niska wartość BLEU w przypadku wszystkich eksperymentów z Transformerem (maksymalnie 17,69%) pokazuje, że mimo poprawnego odwzorowania sensu, model miał trudności z generowaniem dokładnej składni odpowiadającej kodowi referencyjnemu. Model T5-small wykazał znacznie lepsze i bardziej stabilne wyniki. Wartości wszystkich trzech metryk rosły wraz ze wzrostem liczby przykładów w zbiorze treningowym. Już dla 1000 przykładów T5-small osiągnął lepsze wyniki niż Transformer przy 20000. Dla największego zbioru (20000) osiągnięto: Similarity 85,64%, BLEU 14,41% i Semantic Similarity aż 86,11%. Choć BLEU pozostał umiarkowany, podobnie jak w Transformerze, wyraźnie widać, że model był w stanie odtworzyć sens kodu z dużą precyzją, nawet jeśli składnia różniła się od wzorca. Model T5-base, mimo że został przetestowany tylko na zbiorze 1000 przykładów ze względu na ograniczenia obliczeniowe, osiągnął bardzo dobre rezultaty: Similarity 76,87%, BLEU 12,09% i Semantic 74,34%. To pozwala przypuszczać, że przy większym zbiorze T5-base mógłby osiągnąć jeszcze lepsze wyniki niż T5-small, co potwierdza jego większa pojemność i lepsze zdolności reprezentacyjne. Jednak ze względu na długi czas uczenia i ograniczenia sprzętowe nie przeprowadzono jego pełnego skalowania. Podsumowując, najlepsze ogólne rezultaty osiągnął model T5-small trenowany na 20000 przykładach, który wykazał najwyższy poziom zgodności semantycznej (86,11%) i bardzo dobre dopasowanie tekstowe (85,64%). Transformer, mimo dobrej jakości w mniejszych zbiorach, wykazał niestabilność przy większej skali. T5-base potwierdził swój potencjał, ale jego pełne możliwości nie mogły zostać w pełni ocenione w ramach przeprowadzonych eksperymentów.

### **3.4 Wpływ wielkości zbioru danych**

Wielkość zbioru danych treningowych jest jednym z kluczowych czynników determinujących skuteczność modeli językowych w zadaniu dekompilacji kodu maszynowego. Aby przeanalizować tę zależność, przeprowadzono eksperymenty z użyciem czterech różnych wariantów danych: 1000, 5000, 10000 i 20000 przykładów. Modele trenowano zawsze przez trzy epoki, co pozwalało na bezpośrednie porównanie wpływu liczby danych wejściowych przy stałych parametrach treningu. Poniżej przedstawiono wpływ tego czynnika na trzy modele: Transformer, T5-small oraz T5-base (ten ostatni tylko dla jednego wariantu). Dla własnej implementacji modelu Transformer wzrost liczby danych z 1000 do 10000 przykładów przełożył się na wyraźną poprawę jakości. Wskaźnik Similarity wzrósł z 42,39% do 65,83%, a Semantic

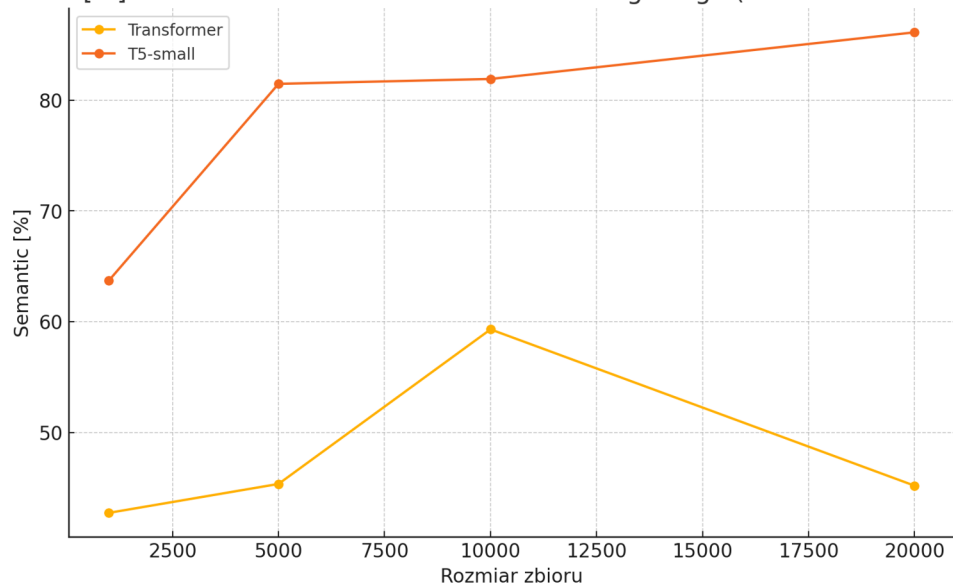
Similarity z 42,74% do 59,30%. Oznacza to, że model lepiej opanował zależności między kodem binarnym a źródłowym, co jest typowe dla architektur uczących się „od zera”. Jednak dla największego zbioru danych – 20000 – wyniki uległy pogorszeniu: Similarity spadło do 43,87%, BLEU do 5,30%, a Semantic również obniżyło się do 45,19%. Ten regres może wynikać z kilku czynników: niedopasowania hiperparametrów (np. zbyt duży learning rate przy większym zbiorze), błędnej kolejności danych w batchach (brak shufflingowania), przeuczenia lub przeciwnie – niemożności dostatecznego dopasowania do szerszego rozkładu danych. Model Transformer nie był pretrenowany, dlatego jego skuteczność zależy silnie od jakości i różnorodności danych oraz od precyzyjnie dobranej architektury. W przypadku T5-small zależność między jakością a rozmiarem danych była bardziej przewidywalna i regularna. Already przy 1000 przykładów model osiągnął 63,67% Similarity i 63,71% Semantic Similarity. Dalszy wzrost liczby danych skutkował coraz lepszymi wynikami: dla 5000 przykładów wartości te wyniosły odpowiednio 83,38% i 81,46%, a przy 10000 osiągnęły 83,60% (Similarity) i 81,90% (Semantic). Co istotne, przy 20000 przykładów wynik Semantic osiągnął poziom 86,11% – najwyższy spośród wszystkich modeli i zbiorów. Warto też zwrócić uwagę na stabilność wskaźnika BLEU, który wzrastał umiarkowanie: od 6,29% (dla 1000) do około 14,4% (dla większych zbiorów). Wynik ten sugeruje, że model nie tylko trafnie odtwarza sens kodu (wysoki Semantic), ale także częściowo jego składnię (BLEU). Regularność przyrostów oraz brak spadków jakości świadczą o dobrej skalowalności architektury T5-small i skuteczności pretrenowania tego modelu. Model T5-base był trenowany tylko na najmniejszym zbiorze (1000 przykładów), z uwagi na bardzo wysokie wymagania sprzętowe i długi czas trenowania (ponad 62 minuty). Mimo to, osiągnął wysoką skuteczność: Similarity 76,87%, BLEU 12,09% i Semantic 74,34%. Są to wyniki zauważalnie lepsze niż te uzyskane przez T5-small przy tej samej liczbie przykładów, co potwierdza, że większe modele mogą szybciej osiągać wysoką jakość przy mniejszych zbiorach dzięki większej pojemności reprezentacyjnej. Niestety, ze względu na ograniczenia techniczne, nie przeprowadzono eksperymentów na większych zbiorach danych.

Similarity [%] w zależności od wielkości zbioru treningowego (Transformer vs. T5-small)



Rys 12: Wykres similarity [%] w zależności od wielkości zbioru treningowego dla modeli Transformer i T5-small.

Semantic [%] w zależności od wielkości zbioru treningowego (Transformer vs. T5-small)



Rys 13: Wykres semantic similarity [%] w zależności od wielkości zbioru treningowego dla modeli Transformer i T5-small

Oba wykresy pokazują, że T5-small wykazuje wyraźną i stabilną poprawę jakości wraz ze wzrostem liczby przykładów, osiągając wartości powyżej 85% w największych zbiorach. Model Transformer natomiast uzyskuje najwyższe wyniki przy 10000 przykładów, po czym jego skuteczność spada — co może sugerować przeuczenie, zbyt małą pojemność modelu lub potrzebę dostrojenia hiperparametrów.

Porównując modele między sobą, łatwo zauważyć, że T5-small wykazuje najlepszy stosunek jakości do liczby danych, oferując wysoką skuteczność i stabilność przy rosnących zbiorach. Transformer jest w stanie dogonić T5 przy odpowiednio dobranych warunkach (np. 10000), ale wykazuje wrażliwość na wielkość danych i ich jakość. T5-base ma potencjał przewyższający oba modele, ale wymaga znacznie większych zasobów. Szczególnie ważne jest to, że T5-small kontynuuje poprawę nawet przy 20000 przykładów, bez oznak przeuczenia — co sugeruje, że model mógłby dalej zyskiwać przy jeszcze większym zbiorze.

### **3.5 Odtwarzanie modelu fizycznego z danych**

W trakcie eksperymentów zaobserwowano interesujące zjawisko: modele językowe – w szczególności T5-small – wykazywały zdolność do poprawnej rekonstrukcji logiki działania programu, nawet jeśli wygenerowany kod odbiegał formalnie od wzorca. Ilustruje to fundamentalną cechę takich systemów: zdolność do odkrywania ukrytej reguły (funkcjonalności), a nie jedynie kopiowania składni. W pewnym sensie działanie to przypomina metodę naukową – model, opierając się na obserwacjach (danych treningowych), wyciąga ogólne wnioski i stosuje je do nowych przypadków.

Przykład 1 – funkcja fib: Wygenerowany kod funkcji obliczającej wartości n-tego elementu ciągu Fibonacciego:

Oczekiwany kod (ground truth):

```
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}

int main() {
    return fib(6);
}
```

Wygenerowany kod:

```
int fib(int n)  if (n = 1) return n; return fib(n - 1) + fib(n - 2);  int
main()  return fib(6);
```

Semantyczne podobieństwo: 99,33%

Model poprawnie uchwycił strukturę rekurencyjną, mimo syntaktycznych błędów. Takie zachowanie wskazuje, że zamiast uczyć się powierzchownego dopasowania, model zrekonstruował regułę: dla  $n \leq 1$  zwróć  $n$ ; w przeciwnym razie zwróć sumę wywołań z mniejszymi argumentami. Jest to przykład odwzorowania dynamicznego procesu, czyli funkcji rekurencyjnej, na podstawie danych – dokładnie tak, jak w fizyce model odkrywa zależność (np. równanie różniczkowe) na podstawie eksperymentalnych pomiarów.

Przykład 2 – funkcja factorial

Oczekiwany kod:

```
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
int main() {
    return factorial(5);
}
```

Wygenerowany kod:

```
int factorial(int n)  if (n = 1) return 1; return n * factorial(n - 1);  int
main()  return factorial(5);
```

Semantyczne podobieństwo: 99,27%

Tu również występuje syntaktyczny błąd ( $n = 1$  zamiast  $n \leq 1$ ), jednak wynikowy kod zachowuje sens działania – zwraca 1 w bazowym przypadku i wykonuje mnożenie w rekurencji. Z fizycznego punktu widzenia, można to potraktować jako przykład modelu, który zidentyfikował "regułę wzrostu wykładniczego" i nauczył się ją stosować. To dowodzi, że model językowy może działać analogicznie do badacza – pomijając detale formalne, rekonstruuje mechanizm ukryty w danych.

Przykład 3 – funkcja power:

Oczekiwany kod:

```
int power(int a, int b) {  
    if (b == 0) return 1;  
    return a * power(a, b - 1);  
}  
  
int main() {  
    return power(5, 2);  
}
```

Wygenerowany kod:

```
int power(int a, int b) if (b == 0) return 1; return a * power(a, b - 1);  
int main() return power(5, 2);
```

Semantyczne podobieństwo: 100,00%

Ten przypadek jest najbardziej precyzyjny – model uchwycił regułę iteracyjnego potęgowania przy pomocy mnożenia i rekurencji. Nie ma tu żadnych zmian wartości parametrów, a struktura jest w pełni zachowana. Jest to bliskie odtworzeniu "prawa fizycznego" – model opisał transformację danych wejściowych na wyjściowe za pomocą poprawnej, przewidywalnej i uniwersalnej reguły.

Te trzy przykłady pokazują, że modele językowe mogą być używane jako narzędzia nie tylko do translacji kodu, ale również do rekonstrukcji logicznych i matematycznych reguł ukrytych w danych – czyli do zadania analogicznego do odkrywania modelu fizycznego na podstawie pomiarów eksperymentalnych. Dodatkowa analiza wyników modelu T5-small trenowanego na pełnym zbiorze 20000 przykładów ujawnia istotne zjawisko: mimo że model bardzo dobrze odwzorowuje strukturę kodu i zachowuje jego semantykę, nadal ma trudności z wiernym odwzorowaniem dokładnych wartości liczbowych użytych w kodzie źródłowym. Problem ten wynika z dwóch czynników. Po pierwsze, sam model ma ograniczoną pojemność – nie został zaprojektowany do dokładnej rekonstrukcji każdej liczby w danym kontekście, a raczej do uchwycenia ogólnej reguły lub struktury logicznej. Po drugie, dane wejściowe (HEX) generowane były losowo, przez co ten sam wzorzec funkcji mógł zawierać zupełnie inne liczby w każdej instancji. Taka zmienność utrudnia nauczanie się precyzyjnego mapowania wartości liczbowych. Mimo to model potrafił zachować poprawność logiczną i poprawnie odwzorować operacje arytmetyczne lub bitowe – nawet jeśli liczby różniły się od wzorcowych. Jest to silna

przesłanka potwierdzająca, że model nauczył się „sensu” programu, nawet jeśli nie znał jego konkretnej formy.

### **3.6 Testowanie na zawężonym zbiorze przypadków**

W celu pogłębienia analizy wpływu struktury zbioru treningowego na jakość generowanego kodu, przeprowadzono dodatkowy eksperyment. Tym razem model T5-small został przetrenowany na zbiorze 20000 przykładów, w którym celowo ograniczono różnorodność — pozostawiając jedynie dwa typy operacji arytmetycznych (np. operacje bitowe i proste działania arytmetyczne), przy jednoczesnym zachowaniu różnorodnych wartości liczbowych. Model został następnie przetestowany na zestawie 50 nowych przykładów, w których występowały analogiczne operacje, lecz z losowo wygenerowanymi parametrami. Celem było sprawdzenie, czy uproszczenie przestrzeni problemu umożliwia modelowi dokładniejsze uchwycenie zależności między kodem maszynowym a jego reprezentacją w języku C. Poniżej przedstawiono wybrane wyniki:

HEX:

ff4300d1ff0f00b948088052e80b00b9e80b8052e80700b9e80b40b9e90740b90001090aff430091c0035fd6

Expected: `int main() { int a = 66, b = 95; return a & b; }`

Generated: `int main() int a = 66, b = 95;return a & b`

HEX: ff4300d1ff0f00b9280a8052e80b00b9e80b40b900310111ff430091c0035fd6

Expected: `int main() { int i = 81; return i + 76; }`

Generated: `int main() int i = 81;return i + 67;`

HEX:

ff4300d1ff0f00b9e8068052e80b00b9e8018052e80700b9e80b40b9e90740b90001090aff430091c0035fd6

Expected: `int main() { int a = 55, b = 15; return a & b; }`

Generated: `int main() int a = 55, b = 15;return a & b;`

Wyniki te są szczególnie istotne z dwóch powodów. Po pierwsze, pomimo syntaktycznych różnic i braku pełnej zgodności formatowania (np. brak średników, nawiasów klamrowych), generowany kod był logicznie poprawny. Po drugie, ograniczenie wariantów logicznych w zbiorze treningowym ułatwiło modelowi nauczenie się zależności funkcjonalnych, co zaowocowało wyższą zgodnością semantyczną. Takie zachowanie potwierdza hipotezę, że model językowy w warunkach uproszczonych potrafi skuteczniej przybliżyć niejawne reguły transformacji — w analogii do procesu odkrywania prawa fizycznego na podstawie

powtarzalnych eksperymentów. Oznacza to, że „przeucenie się” na powtarzalnych strukturach w tym przypadku nie jest wadą, lecz sposobem na uogólnienie konkretnego wzorca logicznego.

### **3.7 Wnioski i obserwacje**

Analiza wyników uzyskanych w eksperymentach przeprowadzonych w ramach niniejszej pracy pozwala sformułować szereg wniosków istotnych zarówno z perspektywy praktycznej, jak i koncepcyjnej. Celem było porównanie skuteczności trzech modeli językowych — własnej implementacji Transformera, modelu T5-small oraz T5-base — w zadaniu dekompilacji kodu maszynowego do języka C. Modele trenowano na zbiorach danych o zróżnicowanej wielkości (1000, 5000, 10000, 20000), a ich skuteczność oceniano przy użyciu trzech niezależnych metryk: Similarity (tekstowe dopasowanie znaków), BLEU (zgodność n-gramów) oraz Semantic Similarity (zgodność logiczna i funkcjonalna wygenerowanego kodu). Najbardziej bezpośrednią obserwacją jest wpływ wielkości zbioru danych treningowych na jakość predykcji. W przypadku modelu Transformer, który trenowany był od zera i nie korzystał z wcześniejszego pretrenowania, poprawa wyników była zauważalna wraz ze wzrostem liczby danych, osiągając maksimum przy 10000 przykładach (Similarity 65,83%, Semantic 59,30%). Jednakże przy 20000 przykładach nastąpił wyraźny spadek jakości — zarówno w metrykach syntaktycznych, jak i semantycznych. Taki regres może wynikać z przeuczenia (overfitting), nadmiernego zróżnicowania danych bez odpowiedniego wzrostu pojemności modelu, niedostosowanego harmonogramu uczenia lub też niedoboru mechanizmów regularyzacyjnych. Warto podkreślić, że własna implementacja modelu Transformer była znacząco mniejsza niż architektury z rodziny T5, co mogło ograniczać jej zdolność do uogólniania przy większych zbiorach. W przeciwieństwie do tego, model T5-small, oparty na architekturze pretrenowanej na dużych zbiorach kodu i języka naturalnego, wykazał znacznie większą odporność na zmianę liczby danych i bardzo dobrą skalowalność. Już przy 1000 przykładach uzyskiwał zadowalające wyniki (Similarity 63,67%, Semantic 63,71%), które rosły konsekwentnie wraz ze wzrostem rozmiaru zbioru. Przy 20000 przykładach osiągnięto najwyższe w całym eksperymencie wartości: Similarity 85,64%, BLEU 14,41% i Semantic Similarity 86,11%. Jest to szczególnie istotne, ponieważ pokazuje, że odpowiednio zaprojektowany i pretrenowany model jest w stanie nie tylko rekonstruować poprawną strukturę kodu, ale także uchwycić jego sens funkcjonalny, co w dekompilacji jest celem nadrzędnym. Z kolei model T5-base, trenowany jedynie na zbiorze 1000 przykładów ze względu na ograniczenia sprzętowe, osiągnął wyniki zauważalnie lepsze niż T5-small przy tym samym rozmiarze danych: Similarity 76,87%, Semantic 74,34%. To potwierdza jego większą pojemność modelową i efektywność uczenia przy ograniczonych danych, ale równocześnie uwidacznia bardzo wysokie koszty obliczeniowe związane z jego dalszym trenowaniem. Mimo braku skalowania na większe zbiory, model ten może być uznany za punkt odniesienia jakościowego, wskazujący na potencjalne możliwości dalszej poprawy wyników przy lepszych zasobach. Jednym z istotnych aspektów zaobserwowanych w



eksperymentach była również różnica w zachowaniu poszczególnych metryk. Similarity oraz BLEU, jako metryki tekstowe, były bardzo czułe na różnice w składni i formatowaniu, co skutkowało zaniżeniem wyników w przypadkach, gdy model wygenerował kod funkcjonalnie poprawny, ale tekstowo odmienny. W przeciwieństwie do tego, Semantic Similarity, wykorzystująca mechanizm normalizacji nazw, wartości liczbowych oraz struktur składniowych, okazała się bardziej odpornym i wiarygodnym miernikiem jakości dekompilacji. W praktyce najlepiej korelowała ona z subiektywnym wrażeniem, czy kod „oddaje” znaczenie oryginału. Metryka ta powinna być traktowana jako podstawowa w kontekście zastosowań dekompileatorów AI. Szczególnie interesującą obserwacją była zdolność modelu T5-small do rekonstruowania funkcji matematycznych i logicznych nawet przy błędach składniowych. Przykłady takie jak `fib`, `factorial` czy `power` pokazały, że model był w stanie uchwycić ogólną regułę przekształcenia danych, mimo że nie generował kodu identycznego ze wzorcem. Można to traktować jako prosty przykład odkrywania modelu fizycznego: AI rekonstruuje „prawa” (czyli logikę programu) z obserwowanych danych wejściowych i wyjściowych, nie znając formalnych zasad ich powstawania. Jest to bliskie idei, że dekompileator AI może pełnić rolę badacza próbującego zrekonstruować ukrytą zależność przyczynowo-skutkową — tak jak w fizyce odkrywamy prawa przyrody na podstawie serii eksperymentów. Dodatkowo, w osobnym eksperymencie przeanalizowano zachowanie modelu w warunkach ograniczonej różnorodności danych. Model T5-small został przetrenowany na 20000 przykładach, w których ograniczono typy operacji do kilku prostych przypadków (np. operacje bitowe i arytmetyczne). Wyniki testów przeprowadzonych na 50 przykładach pokazały średnią semantyczną zgodność na poziomie 87,67%, co potwierdza, że model potrafi skutecznie „wyłapywać” reguły transformacji, gdy przestrzeń rozwiązań jest zawężona. To stanowi kolejne potwierdzenie, że modele językowe mogą działać jak narzędzia statystycznego odkrywania funkcji — ucząc się transformacji nie przez zapisane reguły, lecz przez powtarzające się wzorce. Z punktu widzenia celu pracy — stworzenia dekompileatora wspomagane go sztuczną inteligencją jako minimalistycznego przykładu odkrywania modelu fizycznego — uzyskane wyniki są zgodne z założeniami. Modele oparte na architekturze Transformer są w stanie przybliżyć odwrotność funkcji kompilatora (która formalnie nie istnieje), bazując wyłącznie na przykładach wejście-wyjście, czyli bez jawnej wiedzy o strukturze tej funkcji. Jest to bezpośrednia analogia do sytuacji, w której fizyk — mając do dyspozycji wyłącznie dane eksperymentalne — odkrywa regułę opisującą zjawisko.

**Podsumowując**, wielkość zbioru danych, architektura modelu oraz strategia uczenia mają kluczowe znaczenie dla skuteczności dekompileatora AI. Modele pretrenowane (T5) są zdecydowanie bardziej wydajne i odporne na błędy wynikające z małych zbiorów, podczas gdy modele trenowane od podstaw (Transformer) wymagają ostrożnego dostrajania, ale mogą osiągać konkurencyjne wyniki przy odpowiednim doborze danych i parametrów. Wyniki pracy pokazują, że dekompilacja może być skutecznie modelowana jako problem tłumaczenia sekwencji, a zastosowanie sztucznej inteligencji otwiera drogę do nowych, koncepcyjnych i

praktycznych rozwiązań w dziedzinie inżynierii odwrotnej, jak również stanowi narzędzie ilustracyjne dla zjawiska rekonstruowania reguł — swoistego „odkrywania fizyki” — w domenie informatycznej.

## 4. PODSUMOWANIE.

Niniejsza praca miała na celu zaprojektowanie i przeanalizowanie skuteczności dekompilego wspomagane go sztuczną inteligencją, który przekształca kod maszynowy zapisany w postaci heksadecymalnej na odpowiadający mu kod źródłowy w języku C. Kluczową ideą było potraktowanie dekompilego jako procesu tłumaczenia między językami formalnymi, analogicznie do tłumaczenia języków naturalnych, co umożliwiło zastosowanie architektury Transformer – jednego z najpotężniejszych rozwiązań w dziedzinie modeli językowych. Praca została podzielona na część teoretyczną i praktyczną. W części teoretycznej przedstawiono klasyczne podejścia do inżynierii odwrotnej, omówiono zasady działania tradycyjnych narzędzi dekompilego (takich jak Ghidra, IDA Pro, RetDec czy Snowman), a także wprowadzono podstawy działania modeli językowych Transformer i T5. Szczegółne miejsce poświęcono koncepcji odkrywania modelu fizycznego – metaforze ukazującej proces uczenia maszynowego jako poszukiwanie ukrytej funkcji transformującej dane, analogicznej do praw fizyki wyprowadzanych z eksperymentów. W ramach części praktycznej opracowano kompletny pipeline eksperymentalny. Wygenerowano syntetyczne dane treningowe w postaci par: kod maszynowy (HEX) – odpowiadający mu kod źródłowy (C). Następnie przeprowadzono trening trzech różnych modeli: własnej, lekkiej implementacji modelu Transformer, oraz pretrenowanych modeli T5-small i T5-base. Modele trenowano na zbiorach danych o różnej wielkości (1000, 5000, 10000, 20000 par), a ich skuteczność oceniano za pomocą trzech metryk: Similarity (dopasowanie znaków), BLEU (zgodność n-gramów) oraz Semantic Similarity (zgodność logiczna wygenerowanego kodu). Eksperymenty wykazały wyraźną zależność skuteczności modeli od rozmiaru zbioru treningowego. Model Transformer osiągał najlepsze rezultaty przy zbiorze 10000 przykładów (Semantic Similarity 59,30%), po czym następował spadek jakości, co sugeruje przeuczenie lub niedostosowanie architektury do większej różnorodności danych. Z kolei model T5-small wykazywał dużą stabilność i skalowalność – przy zbiorze 20000 przykładów osiągnięto aż 86,11% zgodności semantycznej, co czyni go najskuteczniejszym w całym eksperymencie. Model T5-base, mimo że trenowany był wyłącznie na 1000 przykładach z powodu ograniczeń sprzętowych, osiągnął bardzo dobre rezultaty – ponad 74% zgodności semantycznej – co dowodzi jego wysokiej pojemności modelowej i efektywności nawet przy ograniczonym treningu. Ważnym elementem analizy była ocena jakości wygenerowanego kodu. W tym celu wprowadzono metrykę Semantic Similarity, która – w przeciwieństwie do klasycznych metryk tekstowych – mierzyła zgodność logiczną i funkcjonalną kodu. Okazała się ona najbardziej wiarygodnym wskaźnikiem przy ocenie użyteczności dekompilego. Przeprowadzone studium przypadków wykazało, że modele są w stanie prawidłowo odwzorować strukturę logiczną nawet w przypadkach, gdy występowały błędy składniowe, brakowało nawiasów czy średników. Szczególnie interesującym rozszerzeniem badań była próba „wytworzenia” modelu fizycznego przez model AI. W jednym z eksperymentów

ograniczono zakres generowanego kodu do kilku klas operacji logicznych i arytmetycznych. Rezultaty pokazały, że model był w stanie uczyć się powtarzalnych wzorców i bardzo dokładnie rekonstruować wartości liczbowe i działania logiczne – osiągając średnią zgodność semantyczną na poziomie prawie 88%. Potwierdza to hipotezę, że model językowy może odgrywać rolę eksperymentatora – rekonstruującego ukryte reguły wyłącznie na podstawie danych wejściowych i wyjściowych. Praca potwierdza, że:

- dekompilacja może być skutecznie realizowana jako proces translacji sekwencji za pomocą modeli językowych,
- modele pretrenowane (T5) są bardziej odporne i efektywne niż modele trenowane od zera (Transformer),
- metryki semantyczne są bardziej miarodajne niż metryki tekstowe przy ocenie jakości kodu,
- architektura transformerów może być wykorzystywana jako narzędzie do badania procesów odwrotnych – również w kontekście filozoficznym, jako ilustracja procesu odkrywania praw przyrody z obserwacji danych.

Wnioski płynące z niniejszej pracy otwierają szerokie perspektywy badawcze – od dalszego rozwoju semantycznych metryk, przez zastosowanie specjalistycznych modeli typu CodeT5, aż po praktyczne wykorzystanie tego podejścia w narzędziach wspomagających analizę binarną i audyt bezpieczeństwa oprogramowania. Co więcej, praca ta pokazuje, że modele językowe mogą pełnić rolę nie tylko translatorów, ale również narzędzi eksploracji wiedzy ukrytej – w duchu statystycznego odkrywania „fizyki programów”.

## WYKAZ LITERATURY.

- [1] Raffel C., Shazeer N., Roberts A., Lee K., Narang S., Matena M., Zhou Y., Li W., Liu P.J.: *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. Journal of Machine Learning Research, Vol. 21, 2020.
- [2] Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A.N., Kaiser Ł., Polosukhin I.: *Attention is All You Need*. Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [3] Kang M., Kim D., et al.: *LLM4Decompilation: Prompting Large Language Models for Binary Decompilation*. ArXiv preprint, 2023.
- [4] Guo J., Zhang Y., et al.: *Deep Learning for Binary Code Analysis: A Survey*. ACM Computing Surveys, 2022.
- [5] Zhang L., Wang Z., et al.: *HexCode: Neural Code Decompiler for Binaries*. In: Proceedings of the 44th IEEE Symposium on Security and Privacy, 2023.
- [6] Hanchen W, Tianfan F, Yuanqi D: *Science in the age of artificial intelligence*. Nature, Vol. 620, 2023, s. 47–57.
- [7] Eilam E.: *Reversing: Secrets of Reverse Engineering*. Wiley Publishing, Indianapolis 2005
- [8] Schwarz B., Debray S.: *Reverse Engineering C++ Applications*. USENIX Workshop on Reverse Engineering, 2002.
- [9] Irfan Ul Haq, Juan Caballero: *A Survey of Binary Code Similarity*. ACM Computing Surveys, Vol. 54, No. 3, 2022.
- [10] Andriesse D. i inni: *An In-Depth Analysis of Disassembly on Full-Scale x86/x64 Binaries*. USENIX Security, 2016.
- [11] Udupa A.V., Debray S., Madou M.: *Deobfuscation: Reverse Engineering Obfuscated Code*. IEEE Working Conference on Reverse Engineering, 2005.
- [12] Hex-Rays Decompiler Documentation, <https://docs.hex-rays.com/>, (data dostępu 08.04.2025 r.)
- [13] Ghidra Software Reverse Engineering Framework, <https://ghidra-sre.org/>, (data dostępu 08.04.2025 r.).
- [14] RetDec Retargetable Decompiler, <https://retdec.com/>, (data dostępu 08.04.2025 r.).
- [15] Snowman Decompiler, <https://derevenets.com/snowman/>, (data dostępu 08.04.2025 r.).
- [16] Chen M. i inni: *Evaluating Large Language Models Trained on Code*. ArXiv:2107.03374, 2021.
- [17] Hugging Face: *Transformers Documentation*. <https://huggingface.co/docs/transformers/index>, (data dostępu: 12.05.2025 r.).
- [18] XYao Y., Duan J., Xu K., Cai Y., Sun Z., Zhang Y.: *A Survey on Large Language Model (LLM) Security and Privacy: The Good, The Bad, and The Ugly*. Department of Computer Science, Drexel University, Philadelphia, PA 19104, USA, 2023.
- [19] Kudo T., Richardson J.: *SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing*. Proceedings of EMNLP 2018.
- [20] Alammr J.: *The Illustrated Transformer*. Blog edukacyjny, 2018.

- [21] NVIDIA: *Transformer Engine User Guide*. NVIDIA Developer Documentation. <https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/>, (data dostępu: 20.04.2025 r.).
- [22] Paszke A. i inni: *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS 2019. <https://pytorch.org>, (data dostępu: 12.05.2025 r.).
- [23] LLVM: *Clang – a C Language Family Frontend for LLVM*. <https://clang.llvm.org>, (data dostępu: 12.05.2025 r.).
- [24] Unix Manual Pages – `otool`, `dd`, `hexdump`. <https://man7.org/linux/man-pages/>, (data dostępu: 12.05.2025 r.).

## WYKAZ RYSUNKÓW

|                                                                                                                                                                                                                                                                                                                                                                 |    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Rys 1: Schemat przedstawia pełen przebieg procesu kompilacji w języku C, włącznie z etapami pośrednimi (.i, .s, .obj) oraz interakcją z plikami nagłówkowymi i bibliotekami .....                                                                                                                                                                               | 11 |
| Rys 2: Schemat procesu dekompilacji: od pliku binarnego, przez deasemblację i analizę kodu, aż do wygenerowania pseudokodu w języku C.....                                                                                                                                                                                                                      | 13 |
| Rys 3: Przykład analizy funkcji main w programie Ghidra.....                                                                                                                                                                                                                                                                                                    | 16 |
| Rys 4: Porównanie popularnych modeli LLM [18].....                                                                                                                                                                                                                                                                                                              | 18 |
| Rys 5: Schemat działania modelu T5 w konkretnym przypadku.....                                                                                                                                                                                                                                                                                                  | 20 |
| Rys 6: Schemat przepływu danych w architekturze dużego modelu językowego (LLM) opartego na Transformerze.....                                                                                                                                                                                                                                                   | 28 |
| Rys 7: Schemat działania dekompilego wspomagane go sztuczną inteligencją. Diagram przedstawia kolejne etapy procesu: od przetwarzania pliku binarnego i ekstrakcji sekcji _text, przez konwersję do formatu HEX i tokenizację, aż po transformację wejściowych danych do postaci kodu źródłowego w języku C za pomocą modelu językowego.....                    | 31 |
| Rys 8: Fragment skryptu w języku Python służącego do automatycznego generowania losowych programów w języku C. Przykłady obejmują pętle (for, while), instrukcje warunkowe (switch-case), operacje bitowe oraz funkcje rekurencyjne (factorial, fibonacci). Kod źródłowy wygenerowany w ten sposób stanowił podstawę zbioru uczącego dla modelu językowego..... | 33 |
| Rys 9: Skrypt bashowy wykorzystywany do kompilacji programów w języku C oraz wyodrębniania sekcji _text z wygenerowanego pliku wykonywalnego. Skrypt automatycznie lokalizuje offset i rozmiar sekcji _text przy użyciu otool, a następnie wycina odpowiadający fragment bajtów i konwertuje go do formatu heksadecymalnego za pomocą hexdump.....              | 34 |
| Rys 10: Przykład przygotowanego batcha danych przekazywanego do modelu językowego.....                                                                                                                                                                                                                                                                          | 40 |
| Rys 11: Schemat przetwarzania danych wejściowych przed przekazaniem do modelu językowego. Tekst w postaci kodu maszynowego (HEX) lub źródłowego (C) podlega tokenizacji, w wyniku której uzyskiwane są sekwencje wejściowe zawierające identyfikatory tokenów (input_ids), maskę uwagi (attention_mask) oraz padding.....                                       | 41 |
| Rys 12: Wykres similarity [%] w zależności od wielkości zbioru treningowego dla modeli Transformer i T5-small.....                                                                                                                                                                                                                                              | 59 |
| Rys 13: Wykres semantic similarity [%] w zależności od wielkości zbioru treningowego dla modeli Transformer i T5-small.....                                                                                                                                                                                                                                     | 59 |

## WYKAZ TABEL

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| Tabela 1: Analogia między procesem fizycznym a trenowaniem dekompiletora AI..... | 9  |
| Tabela 2: Przedstawienie poszczególnych zbiorów danych wraz z ich zastosowaniem. | 38 |
| Tabela 3: Trenowanie modelu Transformer:.....                                    | 54 |
| Tabela 4: Trenowanie modelu T5-small:.....                                       | 55 |
| Tabela 5: Przedstawienie porównawcze wyników trzech modeli:.....                 | 56 |